

MATHEMATICAL FOUNDATIONS OF ALGORITHMS

Rigorous Lecture Notes

A complete theory book for the Spring 2026 examination

Fourier methods, parallel computation, optimization and duality, approximation, randomization, hashing, satisfiability, online algorithms, and spanning trees

Format: monochrome mathematical lecture notes
Coverage: all 26 questions of the supplied examination programme
Prepared for: Timur Saar
Course term: Spring 2026
Edition: July 2026

This text is an independent set of MIT-style lecture notes based on the supplied course material. It is not an official publication of MIT or Saint Petersburg State University.

Contents

I	Fourier Methods and Fast Polynomial Arithmetic	1
1	Roots of Unity, DFT, Convolution, and Polynomial Products	2
1.1	Primitive roots of unity	2
1.2	The direct and inverse DFT	3
1.3	Linear, cyclic, and negacyclic convolution	3
1.4	Karatsuba multiplication	4
1.5	Polynomial multiplication by DFT	5
2	Fast Fourier Transform Algorithms	6
2.1	The computational problem	6
2.2	Decimation in time	6
2.3	Work analysis	7
2.4	Decimation in frequency	7
2.5	The general Cooley-Tukey factorization	7
2.6	The six-step FFT	8
2.7	Inverse FFT and implementation details	8
II	Parallel Circuits and Prefix Computation	10
3	Comparison Circuits and Parallel Sorting	11
3.1	Circuits as a model of parallel computation	11
3.2	Comparator, merging, and sorting networks	11
3.3	The zero-one principle	11
3.4	Batcher's odd-even merging network	12
3.5	Bitonic merging and sorting	13
4	Prefix Accumulation and Its BSP Evaluation	14
4.1	The scan problem	14
4.2	The Ladner-Fischer construction	14
4.3	The BSP model	15
4.4	Evaluating the prefix circuit on BSP	15
5	Parallel Recurrences and Binary Addition	17
5.1	Generalized first-order linear recurrences	17
5.2	Carry-lookahead addition	18

III	Linear Programming, Flows, and Duality	19
6	Linear Programming and Shortest Paths	20
6.1	Linear programs and feasible regions	20
6.2	The simplex idea	21
6.3	Single-source shortest paths as an LP	21
7	Maximum Flow, Augmenting Paths, and Ford-Fulkerson	23
7.1	Flows and the max-flow LP	23
7.2	Residual networks and augmentation	23
7.3	The maximum-flow theorem	24
7.4	The Ford-Fulkerson algorithm	24
8	LP Duality, Complementary Slackness, and Weak Duality	26
8.1	Constructing the dual	26
8.2	Weak duality	26
8.3	Complementary slackness	27
9	Strong Duality	28
9.1	Statement and consequences	28
9.2	Farkas' alternative	28
9.3	Proof of strong duality	28
9.4	Optimality and complementary slackness	29
10	Fractional Cuts, LP Duality, and Integrality by Random Thresholds	30
10.1	The fractional cut dual	30
10.2	Random threshold rounding	31
IV	Approximation Algorithms	32
11	Vertex Cover and LP Rounding	33
11.1	Approximation guarantees	33
11.2	A maximal-matching 2-approximation	33
11.3	The weighted LP relaxation	34
12	The Primal-Dual Method for Weighted Vertex Cover	35
12.1	Primal and dual relaxations	35
12.2	Algorithm	35
13	Metric TSP: Double-Tree and Christofides-Serdyukov	37
13.1	Metric travelling salesperson problem	37
13.2	The double-tree algorithm	37
13.3	The Christofides-Serdyukov algorithm	38
V	Randomized Algebra and Hashing	39
14	Freivalds' Matrix-Product Verification	40
14.1	Verification versus multiplication	40

15 Polynomial Identity Testing and the Schwartz-Zippel Lemma	42
15.1 Identity testing	42
16 Universal Hashing and Multiply-Add	44
16.1 Universal families	44
16.2 The multiply-add family	44
17 Multiply-Shift Hashing	46
17.1 Word-RAM formulation	46
18 Perfect Hashing	47
18.1 One-level perfect hashing	47
18.2 Two-level FKS hashing	47
19 Collision Resolution: Chaining and Linear Probing	49
19.1 Separate chaining	49
19.2 Linear probing	49
20 Binary Probing	51
20.1 Probe order and dyadic blocks	51
20.2 Simplified expected-time analysis	51
VI Satisfiability and Primality	53
21 SAT, k-SAT, and Deterministic 2-SAT	54
21.1 Boolean satisfiability	54
21.2 The implication graph	54
22 A Random-Walk Algorithm for 2-SAT	56
22.1 Algorithm	56
22.2 Hamming-distance analysis	56
23 Schoning's Randomized Algorithm for 3-SAT	58
23.1 Algorithm and local progress	58
23.2 Success probability of one trial	58
24 Miller-Rabin Primality Testing	60
24.1 From Fermat to strong witnesses	60
24.2 Density of witnesses	60
24.3 Complexity and example	61
VII Online Algorithms and Spanning Trees	62
25 Online Algorithms: Scooter Rental and Caching	63
25.1 Competitive analysis	63
25.2 The scooter-rental problem	63
25.3 The paging problem	64
25.4 The randomized marking algorithm	64

26 Minimum Spanning Trees: Boruvka, Prim, and Kruskal	66
26.1 Spanning trees and safe edges	66
26.2 Boruvka's algorithm	66
26.3 Jarnik-Prim's algorithm	67
26.4 Kruskal's algorithm	68
A Examination Syllabus Coverage Index	69
B Reference Sheets and Further Reading	71
B.1 Principal bounds at a glance	71
B.2 Recurring proof templates	72
B.3 Pseudocode and model conventions	72
B.4 Oral-exam checklist	72
B.5 Selected references	73
Closing Perspective	75

Part I

Fourier Methods and Fast Polynomial Arithmetic

Roots of Unity, DFT, Convolution, and Polynomial Products

EXAMINATION SYLLABUS: ITEM 1

The purpose of the Fourier transform in algorithms is not primarily to study waves. It is to change coordinates so that a difficult bilinear operation, convolution, becomes componentwise multiplication. Polynomial multiplication is the central example.

1.1 Primitive roots of unity

Let R be a commutative integral domain with identity. For the algorithms over complex numbers take $R = \mathbb{C}$. We assume that the element represented by the positive integer n is invertible in R .

Definition 1.1 (Primitive root). An element $\omega \in R$ is a **primitive n -th root of unity** if

$$\omega^n = 1, \quad \omega^k \neq 1 \quad (1 \leq k < n).$$

Equivalently, the multiplicative order of ω is n .

Over $\mathbb{C}$, every primitive root has the form $\omega = e^{2\pi i q/n}$ with $\gcd(q, n) = 1$. Its powers $1, \omega, \dots, \omega^{n-1}$ are the vertices of a regular n -gon on the unit circle. Over a finite field the same algebra works whenever an element of order n exists and n is invertible.

Lemma 1.2 (Root-of-unity orthogonality). *For every integer r ,*

$$\sum_{j=0}^{n-1} \omega^{jr} = \begin{cases} n, & n \mid r, \\ 0, & n \nmid r. \end{cases}$$

Proof. If $n \mid r$, every summand is 1. Otherwise $\omega^r \neq 1$, because the order of ω is n . With $S = \sum_{j=0}^{n-1} \omega^{jr}$,

$$(\omega^r - 1)S = \omega^{nr} - 1 = 0.$$

The factor $\omega^r - 1$ is nonzero, and an integral domain has no zero divisors; hence $S = 0$. $\square$

Remark 1.3. In a ring with zero divisors, order n alone need not imply the displayed orthogonality. Number-theoretic transforms therefore select a *principal* root for which the orthogonality relations hold. Over $\mathbb{C}$ this issue never arises.

1.2 The direct and inverse DFT

For $a = (a_0, \dots, a_{n-1})^T \in R^n$, define

$$\hat{a}_k = \sum_{j=0}^{n-1} a_j \omega^{kj}, \quad 0 \leq k < n.$$

The matrix of this map is the Fourier matrix

$$F_{n,\omega} = [\omega^{kj}]_{k,j=0}^{n-1}.$$

Thus $\hat{a} = F_{n,\omega} a$. If $A(x) = \sum_{j=0}^{n-1} a_j x^j$, then $\hat{a}_k = A(\omega^k)$: the DFT is multipoint evaluation on the cyclic group generated by ω .

Theorem 1.4 (Fourier inversion).

$$F_{n,\omega^{-1}} F_{n,\omega} = nI, \quad a_j = \frac{1}{n} \sum_{k=0}^{n-1} \hat{a}_k \omega^{-kj}.$$

Consequently the inverse DFT is $F_{n,\omega}^{-1} = n^{-1} F_{n,\omega^{-1}}$.

Proof. The (r, j) -entry of the matrix product is

$$\sum_{k=0}^{n-1} \omega^{-rk} \omega^{kj} = \sum_{k=0}^{n-1} \omega^{k(j-r)}.$$

By orthogonality this equals n when $r = j$, and 0 otherwise. Multiplying $\hat{a} = F_{n,\omega} a$ by $n^{-1} F_{n,\omega^{-1}}$ gives the inverse formula. $\square$

Worked example 1.5 (A four-point transform). Take $\omega = i$ and $a = (1, 2, 0, 1)$. Direct substitution gives

$$\hat{a} = (4, 1 + i, -2, 1 - i).$$

For instance, $\hat{a}_1 = 1 + 2i + 0 \cdot (-1) + 1 \cdot (-i) = 1 + i$. Applying the inverse formula recovers a . This example is useful for checking the sign convention: a direct transform here uses $+kj$ in the exponent and its inverse uses $-kj$.

The direct definition performs n sums of n terms, so it needs $\Theta(n^2)$ ring operations. The next lecture unit reduces this to $\Theta(n \log n)$ when n factors suitably.

1.3 Linear, cyclic, and negacyclic convolution

Let $a = (a_0, \dots, a_{r-1})$ and $b = (b_0, \dots, b_{s-1})$. Their **linear convolution** is the length- $(r + s - 1)$ vector

$$(a * b)_k = \sum_j a_j b_{k-j},$$

where indices outside their ranges contribute zero. These are exactly the coefficients of $A(x)B(x)$.

Choose $N \geq r + s - 1$, pad both inputs to length N , and let ω be a primitive N -th root.

Theorem 1.6 (Convolution theorem). *For zero-padded inputs,*

$$a * b = \text{DFT}_N^{-1}(\text{DFT}_N(a) \circ \text{DFT}_N(b)),$$

where $\circ$ denotes componentwise multiplication.

Proof. At the k -th Fourier point,

$$\text{DFT}_N(a)_k \text{DFT}_N(b)_k = A(\omega^k)B(\omega^k) = (AB)(\omega^k).$$

The product has degree below N , so its N values determine its coefficient vector uniquely by Fourier inversion. Zero padding is what prevents coefficients of degree at least N from wrapping around. $\square$

For equal length- n vectors, the **cyclic convolution** is

$$(a \circledast b)_k = \sum_{j=0}^{n-1} a_j b_{(k-j) \bmod n}.$$

It is multiplication in $R[x]/(x^n - 1)$. Since ω^k is a root of $x^n - 1$, the same proof gives

$$a \circledast b = \text{DFT}_n^{-1}(\text{DFT}_n(a) \circ \text{DFT}_n(b)).$$

The **negacyclic convolution** is multiplication in $R[x]/(x^n + 1)$. Its coefficients are

$$(a \circledast_- b)_k = \sum_{j=0}^k a_j b_{k-j} - \sum_{j=k+1}^{n-1} a_j b_{n+k-j}.$$

Let ψ be a primitive $2n$ -th root, put $\omega = \psi^2$, and define the twist $T_\psi(a)_j = \psi^j a_j$. The evaluation points $\psi\omega^k$ are precisely the roots of $x^n + 1$. Hence

$$a \circledast_- b = T_\psi^{-1} \left(\text{DFT}_{n,\omega}^{-1} (\text{DFT}_{n,\omega}(T_\psi a) \circ \text{DFT}_{n,\omega}(T_\psi b)) \right).$$

Worked example 1.7 (Why padding matters). For $a = (1, 2)$ and $b = (3, 4)$, linear convolution is $(3, 10, 8)$. A length-two cyclic convolution is $(3 + 8, 10) = (11, 10)$: the coefficient of x^2 wraps to degree zero because $x^2 \equiv 1 \pmod{x^2 - 1}$. Padding to length four recovers the linear result as $(3, 10, 8, 0)$.

1.4 Karatsuba multiplication

Suppose two polynomials have fewer than $n = 2m$ coefficients. Split

$$A = A_0 + x^m A_1, \quad B = B_0 + x^m B_1.$$

Naive divide-and-conquer uses four recursive products. Karatsuba uses three:

$$P_0 = A_0 B_0, \quad P_2 = A_1 B_1, \quad P_1 = (A_0 + A_1)(B_0 + B_1) - P_0 - P_2,$$

and combines them as

$$AB = P_0 + x^m P_1 + x^{2m} P_2.$$

Listing 1.1: Karatsuba polynomial multiplication

```

1 function KARATSUBA(A, B):
2   n = max(length(A), length(B))
3   if n == 1: return [A[0] * B[0]]
4   pad A and B with zeros to the next even length 2m
5   split A = A0 + x^m A1 and B = B0 + x^m B1
6   P0 = KARATSUBA(A0, B0)
7   P2 = KARATSUBA(A1, B1)
8   P1 = KARATSUBA(A0 + A1, B0 + B1) - P0 - P2
9   return P0 + x^m P1 + x^(2m) P2

```

Theorem 1.8 (Karatsuba correctness and complexity). *The algorithm returns AB and uses $\Theta(n^{\log_2 3})$ ring operations.*

Proof. Correctness follows by expanding $(A_0 + x^m A_1)(B_0 + x^m B_1)$; the middle coefficient is $A_0 B_1 + A_1 B_0 = P_1$. Padding changes neither polynomial.

At each non-base call, three problems of half the length are solved and $\Theta(n)$ additions, subtractions, and coefficient moves are performed. Thus

$$T(n) = 3T(n/2) + \Theta(n) = \Theta(n^{\log_2 3})$$

by the recursion-tree sum or the Master theorem. □

1.5 Polynomial multiplication by DFT

For inputs of length at most n , take a transform length N that supports an FFT and satisfies $N \geq 2n - 1$. Over $\mathbb{C}$, the next power of two obeys $N < 4n$.

Listing 1.2: Polynomial multiplication by FFT

```

1 function FFT-MULTIPLY(A, B):
2   N = least power of two with N >= length(A) + length(B) - 1
3   pad A and B to length N
4   FA = FFT(A)
5   FB = FFT(B)
6   for k = 0 to N - 1: FC[k] = FA[k] * FB[k]
7   C = INVERSE-FFT(FC)
8   return the first length(A) + length(B) - 1 coefficients of C

```

Theorem 1.9 (FFT product). *If length- N direct and inverse transforms take $\mathcal{O}(N \log N)$ ring operations, polynomial multiplication takes $\mathcal{O}(n \log n)$ operations. Its auxiliary storage is $\mathcal{O}(n)$.*

Proof. There are two direct transforms, one inverse transform, and N componentwise products. Since $N = \Theta(n)$, their total is $\mathcal{O}(n \log n)$. Correctness is exactly theorem 1.6. □

Common pitfall 1.10. Using a transform of length n for two length- n inputs computes cyclic, not linear, convolution. Also, floating-point FFT requires rounding control; an exact number-theoretic transform avoids this issue when a suitable modulus and root are available.

Fast Fourier Transform Algorithms

EXAMINATION SYLLABUS: ITEM 2

2.1 The computational problem

The input is $a_0, \dots, a_{n-1}$, a primitive n -th root ω , and ring operations. The output is all values $\hat{a}_k = \sum_j a_j \omega^{kj}$. The FFT is a family of algorithms for evaluating this DFT by exploiting a factorization of n . We begin with $n = 2^r$.

2.2 Decimation in time

Separate the input polynomial into its even and odd coefficients:

$$A(x) = A_e(x^2) + xA_o(x^2).$$

Because ω^2 is a primitive $n/2$ -th root,

$$A(\omega^k) = A_e((\omega^2)^k) + \omega^k A_o((\omega^2)^k).$$

For $0 \leq k < n/2$, periodicity gives the butterfly identities

$$\hat{a}_k = E_k + \omega^k O_k, \quad \hat{a}_{k+n/2} = E_k - \omega^k O_k,$$

where $E = \text{DFT}_{n/2, \omega^2}(a_0, a_2, \dots)$ and $O = \text{DFT}_{n/2, \omega^2}(a_1, a_3, \dots)$.

Listing 2.1: Radix-two FFT with decimation in time

```

1 function FFT-DIT(a, omega):
2     n = length(a)
3     if n == 1: return a
4     E = FFT-DIT(a[0], a[2], ..., a[n-2], omega^2)
5     O = FFT-DIT(a[1], a[3], ..., a[n-1], omega^2)
6     for k = 0 to n/2 - 1:
7         t = omega^k * O[k]
8         out[k]      = E[k] + t
9         out[k + n/2] = E[k] - t
10    return out

```

Theorem 2.1 (DIT correctness). *FFT-DITa, omega returns $F_{n, \omega}a$.*

Proof. Induct on n . The case $n = 1$ is immediate. The induction hypothesis computes E and O correctly. The two butterfly identities above then produce every output index k and $k + n/2$, so the returned vector is the DFT. $\square$

The recursive data order repeatedly separates even and odd indices. An in-place iterative implementation therefore begins with a bit-reversal permutation and applies $\log_2 n$ butterfly layers.

Worked example 2.2 (One DIT layer). For $n = 8$, the two recursive inputs are (a_0, a_2, a_4, a_6) and (a_1, a_3, a_5, a_7) . If their transforms are E and O , the four butterflies use twiddles $1, \omega, \omega^2, \omega^3$. Each butterfly creates one output in the lower half and one in the upper half.

2.3 Work analysis

At a size- n call, the butterflies use $\Theta(n)$ operations. Therefore

$$T(n) = 2T(n/2) + \Theta(n).$$

At every recursion level the subproblem lengths sum to n , and there are $\log_2 n$ levels. Hence

$$T(n) = \Theta(n \log n).$$

In the circuit model, all butterflies on a layer are independent, so the circuit has size $\Theta(n \log n)$ and depth $\Theta(\log n)$.

2.4 Decimation in frequency

Decimation in frequency splits the *outputs* into even and odd indices. Put $m = n/2$, and for $0 \leq j < m$ define

$$u_j = a_j + a_{j+m}, \quad v_j = \omega^j(a_j - a_{j+m}).$$

A direct calculation gives

$$\hat{a}_{2k} = \text{DFT}_{m, \omega^2}(u)_k, \quad \hat{a}_{2k+1} = \text{DFT}_{m, \omega^2}(v)_k.$$

Thus the butterflies occur before the recursive transforms. DIT naturally accepts bit-reversed input and returns ordinary output; DIF naturally accepts ordinary input and returns bit-reversed output. Pairing a DIF forward transform with a DIT inverse transform can avoid explicit bit reversal in convolution code.

Theorem 2.3 (DIF complexity). *Radix-two DIF computes the DFT in $\Theta(n \log n)$ operations and has depth $\Theta(\log n)$.*

Proof. The displayed identities prove correctness by the same induction as DIT. Each level performs $n/2$ butterflies and recurses on two inputs of length $n/2$, giving the same recurrence. $\square$

2.5 The general Cooley-Tukey factorization

Let $n = n'n''$. Write the input and output indices as

$$j = n''u + v, \quad k = n's + t,$$

where $0 \leq u, t < n'$ and $0 \leq v, s < n''$. Expanding the exponent gives

$$jk = (n''u + v)(n's + t) \equiv n'sv + tv + n''tu \pmod{n}.$$

Therefore

$$\hat{a}_{n's+t} = \sum_{v=0}^{n''-1} (\omega^{n'})^{sv} \omega^{tv} \left(\sum_{u=0}^{n'-1} (\omega^{n''})^{tu} a_{n''u+v} \right).$$

This formula is the complete reason the general FFT works: first perform length- n' transforms in u , multiply by twiddle factors ω^{tv} , then perform length- n'' transforms in v .

2.6 The six-step FFT

For data stored as a one-dimensional array, the factorization can be arranged as six cache- and parallel-friendly stages:

1. transpose the $n' \times n''$ input matrix;
2. perform n'' independent length- n' transforms;
3. transpose back;
4. multiply entry (t, v) by the twiddle ω^{tv} ;
5. perform n' independent length- n'' transforms;
6. transpose or permute into the required output order.

The exact placement of the three transpositions depends on the chosen row- or column-major convention; the algebraic formula above is invariant.

Theorem 2.4 (General FFT work). *Suppose all prime factors of n are bounded, and base transforms of bounded length take constant work. Recursive Cooley-Tukey, including DIT, DIF, and the six-step organization, uses $\Theta(n \log n)$ ring operations.*

Proof. For $n = n'n''$, one level performs n'' transforms of length n' , n' transforms of length n'' , and $\Theta(n)$ twiddle and permutation work. In a balanced radix decomposition, each element participates in a constant amount of work per level and there are $\Theta(\log n)$ levels. For radix two this reduces to the recurrence already solved. A symmetric split $n' = n'' = \sqrt{n}$ gives $T(n) = 2\sqrt{n}, T(\sqrt{n}) + \Theta(n)$, whose recursion tree also contains $\Theta(\log n)$ total linear-work layers. $\square$

2.7 Inverse FFT and implementation details

Over $\mathbb{C}$, $\omega^{-1} = \bar{\omega}$. Thus an inverse FFT can call the same routine with the conjugate root and divide every output by n . In floating-point code, precompute or update twiddles carefully: repeated multiplication is fast but accumulates phase error, while direct trigonometric evaluation is slower.

Common pitfall 2.5. The formulas do not permit arbitrary reassociation when the coefficient ring is noncommutative. They also require a suitable root at every recursive length. Finally, confusing DIT and DIF data orders is a common source of transforms that contain the right values in the wrong permutation.

Oral-exam note 2.6. For an oral answer, derive one butterfly identity rather than merely drawing the network. Then state the recurrence, explain why every level has linear work, and identify DIT, DIF, and six-step FFT as different organizations of the same Cooley-Tukey factorization.

Part II

Parallel Circuits and Prefix Computation

Comparison Circuits and Parallel Sorting

EXAMINATION SYLLABUS: ITEM 3

3.1 Circuits as a model of parallel computation

Definition 3.1 (Computational circuit). A computational circuit is a directed acyclic graph. Sources are inputs, sinks are outputs, and every internal vertex applies a fixed operation to the values on its incoming wires. Its **size** is the number of operation vertices; its **depth** is the maximum number of operation vertices on an input-to-output path.

Operations on the same depth layer can run simultaneously. Size therefore models total work and depth models ideal parallel time with sufficiently many processors. A circuit is **oblivious**: its graph and the operations to be performed do not depend on the input values. An algorithm for arbitrary input length is a uniformly describable family of circuits $C_1, C_2, \dots$

An FFT circuit has size $\mathcal{O}(n \log n)$, depth $\mathcal{O}(\log n)$, and constant fan-in. This is a useful reminder that a small sequential running time and a small parallel depth are different properties.

3.2 Comparator, merging, and sorting networks

A **comparator** maps two values to

$$(x, y) \mapsto (\min(x, y), \max(x, y)).$$

A **comparison network** consists only of comparators and wires. A **merging network** sorts the union of two already sorted input sequences. A **sorting network** sorts every input sequence.

The network is nonadaptive: it cannot choose its next comparison after seeing an earlier result. This restriction is precisely what permits comparisons on one layer to occur in parallel.

Worked example 3.2 (A small network). Four values can be sorted in depth three: compare wires $(0, 1)$ and $(2, 3)$ in parallel, then $(0, 2)$ and $(1, 3)$, then $(1, 2)$. The network uses five comparators. The zero-one principle below is a convenient way to verify it: only the sixteen binary inputs need be considered.

3.3 The zero-one principle

Theorem 3.3 (Zero-one principle). *A comparison network is a sorting network if and only if it sorts every input from $\{0, 1\}^n$.*

Proof. The forward implication is immediate. For the converse, assume a network fails on some input x over a linearly ordered set. Let its output be y , with $y_k > y_l$ for some $k < l$. Choose a threshold τ satisfying $y_l < \tau \leq y_k$, and define the monotone map

$$\phi(z) = \begin{cases} 0, & z < \tau, \\ 1, & z \geq \tau. \end{cases}$$

Every comparator commutes with ϕ :

$$\phi(\min(u, v)) = \min(\phi(u), \phi(v)), \quad \phi(\max(u, v)) = \max(\phi(u), \phi(v)).$$

Consequently, the network maps the binary input to the binary output

$$(\phi(x_0), \dots, \phi(x_{n-1})) \mapsto (\phi(y_0), \dots, \phi(y_{n-1})).$$

At positions $k < l$ this output contains 1 before 0, so the network also fails on a binary input, a contradiction. $\square$

The same argument verifies merging networks: it is enough to check pairs of sorted binary sequences. A sorted binary sequence is determined solely by its number of zeros.

3.4 Batcher's odd-even merging network

Let $X = (x_0 \leq \dots \leq x_{r-1})$ and $Y = (y_0 \leq \dots \leq y_{s-1})$. Odd-even merge recursively merges the even subsequences and the odd subsequences, interleaves their outputs, and compares the adjacent pairs that can still be inverted.

Listing 3.1: Odd-even merging network

```

1 procedure ODD-EVEN-MERGE(X, Y):
2   if length(X) == 1 and length(Y) == 1:
3     compare-exchange(X[0], Y[0]); return
4   U = ODD-EVEN-MERGE(X[0], X[2], ..., Y[0], Y[2], ...)
5   V = ODD-EVEN-MERGE(X[1], X[3], ..., Y[1], Y[3], ...)
6   interleave U and V
7   parallel compare-exchange(U[i + 1], V[i]) for all valid i

```

Theorem 3.4 (Odd-even merge correctness). *The odd-even merging network correctly merges two sorted sequences.*

Proof. Induct on the total length and use theorem 3.3. Suppose the binary inputs contain r_0 and s_0 zeros. By induction, the recursively merged even-position inputs contain $\lceil r_0/2 \rceil + \lceil s_0/2 \rceil$ leading zeros, while the odd-position inputs contain $\lfloor r_0/2 \rfloor + \lfloor s_0/2 \rfloor$ leading zeros. The two counts differ by at most two. After interleaving, all positions are sorted except possibly one adjacent pattern 10. The final parallel compare-exchange layer changes that pattern to 01. Thus the binary output has exactly $r_0 + s_0$ leading zeros and is sorted. The zero-one principle completes the proof. $\square$

For total length n , its size and depth satisfy

$$M(n) = 2M(n/2) + \mathcal{O}(n) = \mathcal{O}(n \log n), \quad D_M(n) = D_M(n/2) + 1 = \mathcal{O}(\log n).$$

To sort, recursively sort two halves and merge them. Therefore

$$S(n) = 2S(n/2) + \mathcal{O}(n \log n) = \mathcal{O}(n \log^2 n), \quad D_S(n) = D_S(n/2) + \mathcal{O}(\log n) = \mathcal{O}(\log^2 n).$$

3.5 Bitonic merging and sorting

Definition 3.5 (Bitonic sequence). A sequence is bitonic if, after a cyclic shift if necessary, it first nondecreases and then nonincreases. Reversing all inequalities gives the same class.

For $n = 2^r$, a bitonic split compares positions i and $i + n/2$ for all $0 \leq i < n/2$, sending the smaller value to the first half. The operation is then applied recursively to both halves.

Lemma 3.6 (Bitonic split). *After a bitonic split, both halves are bitonic, and every value in the first half is at most every value in the second half.*

Proof. By the zero-one principle it suffices to inspect a bitonic binary sequence. Around the corresponding cycle its zeros form one contiguous block and its ones form the complementary block. Comparing antipodal positions sends every pair 01 to the correct halves. The first half therefore contains only zeros, or a bitonic block ending in ones; the second contains only ones, or a bitonic block beginning with zeros. The number of zeros assigned to the first half is as large as possible, so no one placed there can exceed a zero placed in the second half. The statement for arbitrary ordered values follows from the zero-one principle. $\square$

Theorem 3.7 (Bitonic merge). *Recursive bitonic splitting sorts a bitonic sequence with $\mathcal{O}(n \log n)$ comparators and depth $\mathcal{O}(\log n)$.*

Proof. The split lemma makes the two recursive halves independent and places all values of the first before all values of the second. Induction therefore proves correctness. The size recurrence is $B(n) = 2B(n/2) + n/2$, and the depth recurrence is $D_B(n) = D_B(n/2) + 1$. $\square$

To sort an arbitrary sequence, sort its first half increasingly and its second half decreasingly in parallel; their concatenation is bitonic. Bitonic merge then sorts the whole sequence. Like odd-even merge sort, the resulting network has size $\mathcal{O}(n \log^2 n)$ and depth $\mathcal{O}(\log^2 n)$.

Common pitfall 3.8. The direction of a comparator is a choice of which output wire receives the minimum. A descending subnetwork does not require a new comparison operation; it only swaps the roles of its two output wires.

Prefix Accumulation and Its BSP Evaluation

EXAMINATION SYLLABUS: ITEM 4

4.1 The scan problem

Let $(M, \bullet, e)$ be a monoid: $\bullet$ is associative and e is its identity. Given $a_0, \dots, a_{n-1}$, the inclusive prefix or **scan** problem asks for

$$b_i = a_0 \bullet a_1 \bullet \dots \bullet a_i, \quad 0 \leq i < n.$$

No commutativity is assumed. Prefix sums, prefix products, running minima, function composition, and matrix products are all scans.

The sequential recurrence $b_i = b_{i-1} \bullet a_i$ has optimal linear work but depth $n - 1$. Associativity permits a circuit of linear size and logarithmic depth.

4.2 The Ladner-Fischer construction

For simplicity let n be a power of two. First compute pair products

$$p_i = a_{2i} \bullet a_{2i+1}.$$

Recursively scan $p_0, \dots, p_{n/2-1}$, obtaining q_i . Then

$$b_{2i+1} = q_i, \quad b_{2i} = q_{i-1} \bullet a_{2i} \quad (i > 0), \quad b_0 = a_0.$$

The pair products form an upward layer; the reconstruction of even prefixes forms a downward layer. This is a standard Ladner-Fischer scan organization.

Listing 4.1: Recursive parallel prefix scan

```

1 function PREFIX-SCAN(a[0..n-1]):
2   if n == 1: return [a[0]]
3   parallel for i = 0 to n/2 - 1:
4     p[i] = a[2i] bullet a[2i + 1]
5   q = PREFIX-SCAN(p)
6   b[0] = a[0]
7   parallel for i = 0 to n/2 - 1:
8     b[2i + 1] = q[i]
9     if i > 0: b[2i] = q[i - 1] bullet a[2i]
10  return b

```

Theorem 4.1 (Ladner-Fischer correctness and bounds). *The construction returns all prefixes. It has size at most $2n - 2$ and depth at most $2 \log_2 n$.*

Proof. By induction, $q_i = p_0 \bullet \cdots \bullet p_i$. Associativity gives

$$q_i = (a_0 \bullet a_1) \bullet \cdots \bullet (a_{2i} \bullet a_{2i+1}) = b_{2i+1}.$$

For $i > 0$, $q_{i-1} \bullet a_{2i} = b_{2i}$. Hence every output is correct.

There are at most $n/2$ operations before the recursive call and fewer than $n/2$ afterward, so $W(n) \leq W(n/2) + n$ and $W(n) \leq 2n - 2$. The two parallel layers add at most two to the recursive depth: $D(n) \leq D(n/2) + 2 \leq 2 \log_2 n$. $\square$

Worked example 4.2 (Prefix products of affine functions). Let $f_i(x) = a_i + b_i x$, with composition as the operator. A prefix scan returns $f_0, f_1 \circ f_0, \dots, f_{n-1} \circ \cdots \circ f_0$. Evaluating these functions at one initial value solves every state of a first-order linear recurrence; this is developed in the next unit.

4.3 The BSP model

A bulk-synchronous parallel computer has p processors, each with local memory, and a communication network. A computation is a sequence of **supersteps**. During one superstep processors perform local work and send messages asynchronously; a barrier ends the superstep.

For superstep s , let w_s be the largest local work of any processor and h_s the largest number of words sent or received by any processor. With inverse bandwidth g and latency L , its cost is

$$w_s + gh_s + L.$$

Over the whole algorithm define

$$\text{comp} = \sum_s w_s, \quad \text{comm} = \sum_s h_s, \quad \text{sync} = \#\{\text{supersteps}\},$$

so total cost is $\text{comp} + g \text{comm} + L \text{sync}$.

4.4 Evaluating the prefix circuit on BSP

View the prefix circuit as an upward reduction tree followed by a downward distribution tree. Split each tree into one upper block with p leaves and p lower blocks of size $\Theta(n/p)$. Assign one lower block to each processor. Assuming $n \geq p^2$:

1. every processor reads and reduces its local block;
2. one designated processor scans the p block totals;
3. the resulting offsets are sent back to the processors;
4. every processor completes its local downward pass and writes outputs.

Theorem 4.3 (BSP cost of scan). *Under $n \geq p^2$, prefix accumulation can be evaluated with*

$$\text{comp} = \mathcal{O}(n/p), \quad \text{comm} = \mathcal{O}(n/p), \quad \text{sync} = \mathcal{O}(1).$$

Proof. Each processor owns $\Theta(n/p)$ tree vertices, so its local work is $\mathcal{O}(n/p)$. Reading its input, writing its output, and exchanging one block summary and one offset use $\mathcal{O}(n/p)$ words per processor; this bound also covers a direct transfer between the upward and downward blocks. The four logical stages require only a constant number of barriers. The upper p -leaf tree has $\mathcal{O}(p)$ work on one processor, and $p \leq n/p$ follows from $n \geq p^2$, so it does not violate the local-work bound. $\square$

These bounds are work-optimal and communication-optimal up to constants: the sequential work is $\Theta(n)$, and $\Theta(n)$ input/output words must be distributed across p processors.

Parallel Recurrences and Binary Addition

EXAMINATION SYLLABUS: ITEM 5

5.1 Generalized first-order linear recurrences

Consider

$$c_{-1} = c_{\text{init}}, \quad c_i = a_i \oplus (b_i \odot c_{i-1}).$$

Assume $\oplus$ and $\odot$ are associative, $\odot$ distributes on the left over $\oplus$, and their elementary operations take constant time. The ordinary recurrence uses $+$ $= \oplus$ and multiplication $= \odot$; shortest path recurrences use $(\min, +)$; Boolean recurrences may use $(\vee, \wedge)$.

Associate with (a, b) the affine transformation $T_{a,b}(x) = a \oplus (b \odot x)$. Composition has the closed form

$$T_{a,b} \circ T_{a',b'} = T_{a \oplus (b \odot a'), b \odot b'}.$$

Lemma 5.1 (Associative affine product). *The binary operation*

$$(a, b) \star (a', b') = (a \oplus (b \odot a'), b \odot b')$$

is associative.

Proof. The pair (a, b) represents the function $T_{a,b}$, and $\star$ represents function composition. Function composition is associative. The distributive law proves that the displayed pair indeed represents the composed function, so $\star$ is associative on these pairs. $\square$

Scanning the transformations in their application order gives

$$T_i \circ \cdots \circ T_0,$$

and evaluating each prefix transformation at c_{-1} yields every c_i . The scan therefore has size $\mathcal{O}(n)$ and depth $\mathcal{O}(\log n)$.

For ordinary arithmetic one may equivalently use matrices:

$$\begin{pmatrix} 1 & 0 \\ a_i & b_i \end{pmatrix} \begin{pmatrix} 1 \\ c_{i-1} \end{pmatrix} = \begin{pmatrix} 1 \\ c_i \end{pmatrix}.$$

All prefix matrix products are another instance of the same scan.

5.2 Carry-lookahead addition

Let $x_i, y_i \in \{0, 1\}$ be bits listed from least significant to most significant. Define the generate and propagate bits

$$g_i = x_i \wedge y_i, \quad p_i = x_i \oplus y_i.$$

With incoming carry $c_{-1} = 0$,

$$c_i = g_i \vee (p_i \wedge c_{i-1}), \quad z_i = p_i \oplus c_{i-1}, \quad z_n = c_{n-1}.$$

The carry equation is a generalized linear recurrence with $\oplus = \vee$ and $\odot = \wedge$.

For two bit positions or blocks, the combined generate/propagate pair is

$$(g, p) \star (g', p') = (g \vee (p \wedge g'), p \wedge p'),$$

with the order chosen to match the direction from low to high bits. A prefix scan computes every carry in logarithmic depth; all sum bits are then computed in one additional layer.

Theorem 5.2 (Parallel binary addition). *Two n -bit numbers can be added by a bounded-fan-in Boolean circuit of size $\mathcal{O}(n)$ and depth $\mathcal{O}(\log n)$.*

Proof. Computing all g_i, p_i has linear size and constant depth. The associative prefix circuit on their pairs has linear size and logarithmic depth. Each output bit is one XOR of a propagate bit with its incoming carry, adding one layer and linear size. The carry recurrence proves correctness. $\square$

Worked example 5.3 (Adding $1011_2 + 0110_2$). From least significant to most significant,

$$x = (1, 1, 0, 1), \quad y = (0, 1, 1, 0).$$

Thus $g = (0, 1, 0, 0)$, $p = (1, 0, 1, 1)$, and the carry prefixes are $c = (0, 1, 1, 1)$. The output bits are $(1, 0, 0, 0, 1)$, so the result is $10001_2 = 17$.

Common pitfall 5.4. Composition is associative but generally not commutative. Reversing the scan order solves a different recurrence. In binary addition, also distinguish the propagate convention $x_i \oplus y_i$ from the alternative $x_i \vee y_i$; both can be made correct, but their block-combination formulas must not be mixed.

Part III

Linear Programming, Flows, and Duality

Linear Programming and Shortest Paths

EXAMINATION SYLLABUS: ITEM 6

6.1 Linear programs and feasible regions

A linear program optimizes a linear function over points satisfying linear equalities and inequalities. A maximization problem can be written in the general form

$$\max c^T x \quad \text{subject to} \quad a_i^T x \square_i b_i,$$

where each $\square_i$ is $\leq$, $=$, or $\geq$, and variables may have sign restrictions.

Definition 6.1 (Feasibility and optimality). A vector is **feasible** if it satisfies every constraint. A feasible vector x^* is **optimal** for a maximization LP if $c^T x \leq c^T x^*$ for every feasible x . An LP is **infeasible** if it has no feasible point, and **unbounded** if feasible objective values have no finite upper bound.

The **standard inequality form** used in these notes is

$$\max c^T x \quad \text{subject to} \quad Ax \leq b, \quad x \geq 0. \tag{P}$$

The **equational form** is

$$\max c^T x \quad \text{subject to} \quad Ax = b, \quad x \geq 0.$$

Any LP can be converted to either form:

- minimizing $c^T x$ is maximizing $-c^T x$;
- $a^T x \geq b$ becomes $-a^T x \leq -b$;
- an equality is the conjunction of two opposite inequalities;
- a free variable is replaced by $x = x^+ - x^-$, with $x^+, x^- \geq 0$;
- $Ax \leq b$ becomes $Ax + s = b$ using slack variables $s \geq 0$.

The feasible set is an intersection of half-spaces and hyperplanes, hence a convex polyhedron. Its vertices are its extreme points.

Theorem 6.2 (Fundamental theorem of linear programming). *If an LP is feasible and bounded, it has an optimal solution. If it has an optimal solution and its feasible polyhedron has a vertex, then some optimal solution is a vertex.*

As required by the examination programme, this theorem is used without proof. It explains why vertex-to-vertex algorithms are meaningful, although a polyhedron may have exponentially many vertices.

Worked example 6.3 (A two-product LP). Suppose products 1, 2 yield profits 3, 2, and two resources impose

$$x_1 + x_2 \leq 4, \quad 2x_1 + x_2 \leq 5, \quad x_1, x_2 \geq 0.$$

The objective is $3x_1 + 2x_2$. The intersection of the two active resource constraints is $(1, 3)$, with value 9. The other nonzero axis vertices have values 8 and 7.5, so $(1, 3)$ is optimal. Later, the dual vector $(1, 1)$ will certify this without enumerating vertices.

6.2 The simplex idea

In equational form, a choice of linearly independent columns of A is a **basis**. Solving for the associated basic variables produces a basic solution; when it is nonnegative, it represents a vertex. The simplex method:

1. starts at a feasible basis;
2. computes reduced costs, which describe whether an adjacent edge can improve the objective;
3. chooses an entering variable with improving reduced cost;
4. uses a ratio test to choose a leaving variable while preserving feasibility;
5. pivots to the adjacent basis and repeats.

If no improving reduced cost exists, the current basic feasible solution is optimal and the reduced costs yield a dual certificate. Degeneracy can cause a pivot without objective improvement and even cycling; deterministic pivot rules such as Bland's rule prevent cycling. Simplex is extremely effective in practice but has exponential worst-case examples. Ellipsoid and interior-point methods give polynomial-time algorithms for LP.

6.3 Single-source shortest paths as an LP

Let $G = (V, E)$ be directed, let $s \in V$, and let edge weights $w : E \rightarrow \mathbb{R}$. Assume every vertex is reachable from s and no reachable negative cycle exists. Let $\delta(v)$ be the shortest-path distance from s to v . Consider

$$\max \sum_{v \in V} x_v \quad \text{subject to} \quad x_s = 0, \quad x_v - x_u \leq w(u, v) \quad ((u, v) \in E). \quad (\text{SP-LP})$$

Lemma 6.4 (Every feasible potential is a lower bound). *If x is feasible for (SP-LP), then $x_v \leq \delta(v)$ for every v .*

Proof. Let $s = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k = v$ be any s -to- v path. Summing its constraints gives

$$x_v - x_s \leq \sum_{i=1}^k w(v_{i-1}, v_i).$$

Because $x_s = 0$, x_v is at most the weight of every such path, hence at most their minimum $\delta(v)$. $\square$

Theorem 6.5 (Shortest-path LP correctness). *The vector $x_v = \delta(v)$ is the unique componentwise maximum feasible solution and is optimal for (SP-LP).*

Proof. We have $\delta(s) = 0$, and the triangle inequality for shortest paths gives $\delta(v) \leq \delta(u) + w(u, v)$ for every edge. Thus δ is feasible. The lemma says every feasible x satisfies $x_v \leq \delta(v)$ componentwise, so its objective is at most $\sum_v \delta(v)$. Equality in the sum forces equality in every component. $\square$

If a vertex is unreachable, its variable can grow without bound. If a reachable negative cycle exists, the distance model loses finite meaning and the constraint system becomes inconsistent after summing around that cycle.

Common pitfall 6.6. The LP does not itself imply that a generic LP solver is the best shortest-path algorithm. Its value is conceptual and structural: it exposes distance labels as potentials and makes dual certificates available for modified variants.

Maximum Flow and Augmenting Paths

Ford-Fulkerson Algorithm

EXAMINATION SYLLABUS: ITEM 7

7.1 Flows and the max-flow LP

Let $G = (V, E)$ be directed, with source s , sink t , and capacities $c_e \geq 0$. A flow $f : E \rightarrow \mathbb{R}_{\geq 0}$ satisfies

$$0 \leq f_e \leq c_e$$

and conservation at every $v \notin \{s, t\}$: inflow equals outflow. Its value is the net outflow from s , equivalently the net inflow to t .

The maximum-flow problem is the LP

$$\max |f| \quad \text{subject to conservation and} \quad 0 \leq f_e \leq c_e.$$

It is feasible because the zero flow is feasible, and bounded by the total capacity leaving s .

For $S \subseteq V$ with $s \in S$, $t \notin S$, the directed cut capacity is

$$c(S, V \setminus S) = \sum_{u \in S, v \notin S} c(u, v).$$

Lemma 7.1 (Flow across a cut). *For every feasible flow and every s - t cut (S, T) ,*

$$|f| = \sum_{u \in S, v \in T} f(u, v) - \sum_{u \in T, v \in S} f(u, v) \leq c(S, T).$$

Proof. Sum net outflow over vertices of S . Contributions of edges internal to S cancel. Conservation cancels every vertex except s , giving the identity. Drop the nonnegative backward-flow term and apply capacities on forward edges to obtain the inequality. $\square$

7.2 Residual networks and augmentation

For each original edge $e = (u, v)$, the residual graph contains

- a forward residual edge $u \rightarrow v$ of capacity $c_e - f_e$ if positive;
- a reverse residual edge $v \rightarrow u$ of capacity f_e if positive.

A residual s -to- t path is an **augmenting path**. Its bottleneck Δ is the minimum residual capacity on the path. Add Δ along forward original edges and subtract it along reverse original edges.

Lemma 7.2 (Augmentation). *Augmenting by Δ preserves feasibility and increases the flow value by Δ .*

Proof. The bottleneck definition prevents any capacity or nonnegativity violation. At each internal path vertex, one path edge enters and one leaves, so the same amount is added to inflow and outflow. At s net outflow rises by Δ , and at t net inflow rises by Δ . $\square$

7.3 The maximum-flow theorem

Theorem 7.3 (Augmenting-path and max-flow/min-cut theorem). *For a feasible flow f , the following are equivalent:*

1. f is maximum;
2. the residual graph has no augmenting path;
3. some s - t cut has capacity $|f|$.

Consequently, maximum flow value equals minimum cut capacity.

Proof. If an augmenting path exists, the augmentation lemma produces a larger flow, so 1 implies 2. If a cut has capacity $|f|$, the cut upper bound proves that no larger flow exists, so 3 implies 1.

Assume 2. Let S be the vertices reachable from s in the residual graph and $T = V \setminus S$. Then $t \in T$. Every original edge from S to T is saturated; otherwise its forward residual edge would reach T . Every original edge from T to S carries zero flow; otherwise its reverse residual edge would reach its tail in T . The cut-flow identity now gives

$$|f| = \sum_{S \rightarrow T} c_e - 0 = c(S, T),$$

which is 3. Taking an optimal flow and the resulting cut proves equality of the two optimum values. $\square$

7.4 The Ford-Fulkerson algorithm

Listing 7.1: Ford-Fulkerson with an arbitrary path search

```

1 function FORD-FULKERSON(G, c, s, t):
2     f[e] = 0 for every edge e
3     while residual graph G_f contains an s-to-t path P:
4         delta = minimum residual capacity on P
5         for each residual edge on P:
6             if it is forward for original edge e: f[e] += delta
7             else for original edge e:             f[e] -= delta
8     return f

```

Theorem 7.4 (Ford-Fulkerson correctness). *If the algorithm terminates, it returns a maximum flow and an associated minimum cut.*

Proof. The augmentation lemma maintains a feasible flow. At termination no residual augmenting path remains, so theorem 7.3 proves maximality. Residual reachability from s produces a cut of equal value. $\square$

Theorem 7.5 (Integral-capacity bound). *If capacities and the initial flow are integers and F^* is the maximum flow value, Ford-Fulkerson starting from zero performs at most F^* augmentations and runs in $\mathcal{O}(|E|F^*)$ time with a linear-time DFS or BFS path search.*

Proof. All residual capacities remain integers. Every positive bottleneck is therefore at least one, so every iteration raises the integer flow value by at least one. It starts at zero and never exceeds F^* . Constructing or searching the residual adjacency structure and updating a path costs $\mathcal{O}(|E|)$ per iteration. $\square$

The bound is pseudo-polynomial: F^* can be exponentially large compared with the binary length of the capacities. With arbitrary irrational capacities, unfortunate path choices can even produce a nonterminating sequence. Algorithms such as Edmonds-Karp and Dinic choose paths more carefully and have polynomial bounds independent of capacity magnitudes.

Worked example 7.6 (Why reverse residual edges matter). Suppose an early augmentation sends one unit through $s \rightarrow a \rightarrow b \rightarrow t$, but the final optimum instead needs that unit on $s \rightarrow a \rightarrow t$ and another on $s \rightarrow b \rightarrow t$. A later residual path can traverse $b \rightarrow a$, the reverse of $a \rightarrow b$, canceling the earlier decision. A search restricted to unsaturated forward edges would incorrectly become stuck.

LP Duality, Complementary Slackness, and Weak Duality

EXAMINATION SYLLABUS: ITEM 8

8.1 Constructing the dual

For the standard-form primal

$$\max c^T x \quad \text{subject to} \quad Ax \leq b, \quad x \geq 0, \quad (\text{P})$$

the dual is

$$\min b^T y \quad \text{subject to} \quad A^T y \geq c, \quad y \geq 0. \quad (\text{D})$$

There is one dual variable per primal general constraint and one dual constraint per primal variable. The matrix is transposed; right-hand sides and objective coefficients exchange roles.

For an equational primal

$$\max c^T x \quad \text{subject to} \quad Ax = b, x \geq 0,$$

the dual remains $\min b^T y$ with $A^T y \geq c$, but y is free. An equality constraint corresponds to a free dual variable. Conversely, a free primal variable corresponds to an equality dual constraint.

One way to remember the construction is to multiply the primal inequalities by nonnegative prices y_i . If $A^T y \geq c$, then the resulting weighted sum dominates the objective coefficient by coefficient.

8.2 Weak duality

Theorem 8.1 (Weak duality). *If x is feasible for (P) and y is feasible for (D), then*

$$c^T x \leq b^T y.$$

Proof. Because $x, y \geq 0$,

$$c^T x \leq (A^T y)^T x = y^T Ax \leq y^T b = b^T y.$$

The first inequality uses $A^T y \geq c$, and the second uses $Ax \leq b$. □

Thus every dual feasible point is an upper-bound certificate for every primal feasible point. If a feasible pair has equal values, both are optimal.

8.3 Complementary slackness

For feasible x, y , define primal slack $b - Ax \geq 0$ and dual slack $A^T y - c \geq 0$. The complementary-slackness conditions are

$$\begin{aligned} y_i(b_i - (Ax)_i) &= 0 \quad \text{for every } i, \\ x_j((A^T y)_j - c_j) &= 0 \quad \text{for every } j. \end{aligned}$$

In words, a positive variable can correspond only to a tight opposite constraint. The converse is not required: a tight constraint may still have a zero corresponding variable.

Lemma 8.2 (Complementary slackness certificate). *If primal and dual feasible vectors satisfy complementary slackness, then $c^T x = b^T y$, and hence both vectors are optimal.*

Proof. The duality gap decomposes as

$$b^T y - c^T x = y^T(b - Ax) + x^T(A^T y - c).$$

Every summand on the right is nonnegative. Complementary slackness makes each componentwise product zero, so the gap is zero. Weak duality then certifies optimality. $\square$

Worked example 8.3 (A dual certificate). For the two-product LP from the previous unit, the dual is

$$\min 4y_1 + 5y_2, \quad y_1 + 2y_2 \geq 3, \quad y_1 + y_2 \geq 2, \quad y \geq 0.$$

The vectors $x = (1, 3)$ and $y = (1, 1)$ are feasible and both objectives equal 9. Every primal resource constraint and every dual product constraint is tight, so complementary slackness is immediate. This proves optimality without drawing the feasible polygon.

Common pitfall 8.4. Weak duality does not say that every feasible primal-dual pair has equal values. It says only that the dual value is an upper bound for a maximization primal. Equality is the optimality certificate.

Strong Duality

EXAMINATION SYLLABUS: ITEM 9

9.1 Statement and consequences

Theorem 9.1 (Strong duality). *If the standard-form primal is feasible and has a finite optimum v^* , then its dual has an optimum y^* and*

$$c^T x^* = v^* = b^T y^*.$$

Equivalently, when both programs are feasible, their optimum values are equal.

The proof below is valid in degenerate cases and makes the certificate nature of duality explicit. It uses one geometric alternative.

9.2 Farkas' alternative

Lemma 9.2 (Farkas). *For a real matrix B and vector d , exactly one of the following systems has a solution:*

$$\begin{aligned} Bz = d, \quad z \geq 0; \\ q^T B \geq 0, \quad q^T d < 0. \end{aligned}$$

Proof. They cannot both hold, because then $q^T d = q^T Bz \geq 0$.

Suppose d is not in the finitely generated closed convex cone $K = \{Bz : z \geq 0\}$. Let $p \in K$ minimize the Euclidean distance to d , and put $q = p - d$. For any $k \in K$, the ray $p + tk$ lies in K for $t \geq 0$. The one-sided derivative at the minimizer gives $q^T k \geq 0$. In particular, $q^T B \geq 0$ columnwise. Since the whole line segment $(1+t)p$ remains in K for $t \geq -1$, derivatives in both directions at zero give $q^T p = 0$. Therefore

$$q^T d = q^T (p - q) = -\|q\|^2 < 0,$$

because $d \notin K$ implies $q \neq 0$. Thus the second system has a solution. □

9.3 Proof of strong duality

Proof of theorem 9.1. Let v^* be the finite primal optimum. We seek nonnegative vectors y, s and a scalar $r \geq 0$ satisfying

$$A^T y - s = c, \quad b^T y + r = v^*. \tag{9.1}$$

Such a solution makes y dual feasible and gives $b^T y \leq v^*$.

Assume for contradiction that 9.1 has no solution. Apply Farkas' lemma to its coefficient matrix and right-hand side (c, v^*) . There exist $u \in \mathbb{R}^n$ and $\lambda \in \mathbb{R}$ such that the columns for y, s, r give

$$Au + \lambda b \geq 0, \quad -u \geq 0, \quad \lambda \geq 0,$$

while

$$c^T u + \lambda v^* < 0.$$

Put $z = -u \geq 0$. Then

$$Az \leq \lambda b, \quad c^T z > \lambda v^*.$$

If $\lambda > 0$, the vector z/λ is primal feasible and has objective strictly greater than v^* , a contradiction. If $\lambda = 0$, then $Az \leq 0$ and $c^T z > 0$. Starting from any primal feasible x_0 , every $x_0 + tz$, $t \geq 0$, remains feasible and its objective tends to infinity, again a contradiction.

Hence 9.1 is solvable. Its y is dual feasible and $b^T y \leq v^*$. Weak duality applied to a primal optimum gives $v^* \leq b^T y$, so equality holds. Thus $r = 0$, and y is dual optimal. $\square$

9.4 Optimality and complementary slackness

For any feasible x, y ,

$$b^T y - c^T x = \sum_i y_i (b_i - (Ax)_i) + \sum_j x_j ((A^T y)_j - c_j).$$

All terms are nonnegative. Strong duality says an optimal pair has zero gap, so every term is zero. Conversely, complementary slackness makes the gap zero and weak duality proves optimality. We have therefore proved:

Corollary 9.3 (Three equivalent optimality conditions). *For feasible primal-dual vectors x, y , the following are equivalent:*

1. x, y are both optimal;
2. $c^T x = b^T y$;
3. x, y satisfy complementary slackness.

Proof. Strong duality gives $1 \Rightarrow 2$, while weak duality makes equality of feasible objective values sufficient for optimality, so $2 \Rightarrow 1$. The nonnegative gap decomposition above vanishes exactly when every complementary product vanishes; hence $2 \Leftrightarrow 3$. $\square$

Oral-exam note 9.4. The short oral structure is: weak duality gives a nonnegative gap; Farkas produces a dual point with value no larger than the primal optimum; the two bounds force equality; the gap decomposition yields complementary slackness.

Fractional Cuts and LP Duality

Integrality by Random Thresholds

EXAMINATION SYLLABUS: ITEM 10

10.1 The fractional cut dual

Introduce a potential z_v for every vertex, normalized by $z_s = 1$, $z_t = 0$, and a nonnegative length y_e for each directed edge. The fractional minimum-cost cut LP is

$$\min \sum_{e=(u,v)} c_e y_e \quad (\text{FC})$$

subject to

$$y_{uv} \geq z_u - z_v, \quad y_{uv} \geq 0.$$

The potentials of internal vertices are free; an optimum may be clipped to $[0, 1]$. For fixed z , the cheapest edge variable is

$$y_{uv} = \max\{z_u - z_v, 0\}.$$

This LP is the dual of maximum flow. Conservation equalities produce the free vertex potentials, and capacity upper bounds produce the nonnegative edge variables. A direct weak-duality calculation makes the relationship clear.

Lemma 10.1 (Flow versus fractional cut). *For every feasible flow f and fractional cut (y, z) ,*

$$|f| \leq \sum_e c_e y_e.$$

Proof. Flow conservation telescopes against the potentials:

$$|f| = \sum_{e=(u,v)} f_e (z_u - z_v).$$

Indeed, the coefficient of z_v is net outflow at v , which is zero internally, $+|f|$ at s , and $-|f|$ at t . Now use $z_u - z_v \leq y_e$, $f_e \geq 0$, and $f_e \leq c_e$:

$$|f| \leq \sum_e f_e y_e \leq \sum_e c_e y_e.$$

□

Strong duality therefore identifies the optimum value of (FC) with the maximum-flow value. It remains to see why the fractional optimum can be made an actual cut.

10.2 Random threshold rounding

Clipping every z_v to the interval $[0, 1]$ does not increase any positive difference $\max(z_u - z_v, 0)$, so assume $0 \leq z_v \leq 1$. Choose τ uniformly from $[0, 1]$ and define

$$S_\tau = \{v : z_v \geq \tau\}, \quad T_\tau = V \setminus S_\tau.$$

Except on a probability-zero endpoint event, $s \in S_\tau$ and $t \in T_\tau$, so this is an s - t cut.

Lemma 10.2 (Crossing probability). *For $e = (u, v)$,*

$$\mathbb{P}(u \in S_\tau, v \in T_\tau) = \max\{z_u - z_v, 0\} \leq y_e.$$

Proof. The edge crosses exactly when $z_v < \tau \leq z_u$. The probability is the length of this interval inside $[0, 1]$, namely its positive difference. Feasibility gives the final inequality. $\square$

Theorem 10.3 (Integrality of the fractional cut optimum). *Every feasible fractional cut can be rounded to an integral s - t cut of no greater cost. In particular, (FC) has an integral optimum.*

Proof. By linearity of expectation and the crossing lemma,

$$\mathbb{E}[c(S_\tau, T_\tau)] = \sum_e c_e \mathbb{P}(e \text{ crosses}) \leq \sum_e c_e y_e.$$

Some threshold has cost at most the expectation. Its characteristic potentials $z'_v \in \{0, 1\}$ and crossing indicators $y'_{uv} = \max(z'_u - z'_v, 0)$ form an integral feasible solution of no greater cost. Rounding an optimal fractional solution therefore produces an integral optimum. $\square$

Combining integrality with strong duality yields max-flow/min-cut once more:

$$\max\{\text{flow value}\} = \min\{\text{fractional cut cost}\} = \min\{\text{cut capacity}\}.$$

Worked example 10.4 (Thresholds expose several cuts). Suppose $z_s = 1$, $z_a = 0.7$, $z_b = 0.3$, and $z_t = 0$. Thresholds in $(0.7, 1]$, $(0.3, 0.7]$, and $(0, 0.3]$ produce respectively $\{s\}$, $\{s, a\}$, and $\{s, a, b\}$ on the source side. The expected cut cost is the weighted average of these three integral cuts, which cannot be smaller than all of them. Thus at least one is no more expensive than the fractional solution.

Remark 10.5. This is the probabilistic method in algorithmic form: a random object is used to prove that a good deterministic object exists. The rounding can be derandomized by sorting the finitely many distinct potential values and checking the corresponding threshold cuts.

Part IV

Approximation Algorithms

Vertex Cover and LP Rounding

EXAMINATION SYLLABUS: ITEM 11

11.1 Approximation guarantees

For a minimization problem, a polynomial-time algorithm is a ρ -**approximation**, $\rho \geq 1$, if it always returns a feasible solution of value at most ρOPT . For a maximization problem the returned value must be at least OPT/ρ . The guarantee is worst-case and compares solutions on the same input.

In the vertex-cover problem, an undirected graph $G = (V, E)$ is given, and we seek $S \subseteq V$ meeting every edge. The unweighted objective minimizes $|S|$; the weighted objective minimizes $c(S) = \sum_{v \in S} c_v$ for nonnegative costs.

11.2 A maximal-matching 2-approximation

Choose any uncovered edge, put both endpoints in the cover, and delete all edges incident to either endpoint. The chosen edges form a maximal matching.

Listing 11.1: Unweighted vertex cover from a maximal matching

```

1 function APPROX-VERTEX-COVER(G):
2   S = empty set
3   mark every edge uncovered
4   while an uncovered edge (u, v) exists:
5     add u and v to S
6     mark every edge incident to u or v covered
7   return S

```

Theorem 11.1 (Maximal-matching approximation). *The algorithm returns a vertex cover of size at most 2OPT in $\mathcal{O}(|V| + |E|)$ time.*

Proof. At termination no edge has both endpoints outside S , so S is a cover. Let M be the selected edges. They share no endpoints, because all edges incident to a chosen endpoint are removed before the next selection. Every vertex cover must contain at least one endpoint of each edge in M ; since these edges are disjoint, $\text{OPT} \geq |M|$. The algorithm selects exactly two vertices per edge, so

$$|S| = 2|M| \leq 2 \text{OPT}.$$

An adjacency-list implementation processes each edge only a constant number of times. □

Common pitfall 11.2. The proof uses cardinality. With arbitrary vertex costs, taking both endpoints of a maximal matching may be arbitrarily bad: one endpoint may be enormously more expensive than a cheap neighbor that covers all the same selected edges.

11.3 The weighted LP relaxation

The integer program has a variable $x_v \in \{0, 1\}$ for each vertex:

$$\min \sum_{v \in V} c_v x_v, \quad x_u + x_v \geq 1 \quad (uv \in E).$$

Relax integrality to $x_v \geq 0$. Its optimum LP^* is a lower bound on the minimum integral cover OPT.

Round an optimal fractional solution by

$$\bar{x}_v = \begin{cases} 1, & x_v^* \geq 1/2, \\ 0, & x_v^* < 1/2. \end{cases}$$

Theorem 11.3 (LP-rounding approximation). *Threshold rounding at $1/2$ returns a weighted vertex cover of cost at most 2 OPT .*

Proof. For each edge uv , feasibility gives $x_u^* + x_v^* \geq 1$, so at least one endpoint is at least $1/2$; the rounded set covers the edge. Moreover $\bar{x}_v \leq 2x_v^*$ for every vertex. Hence

$$\sum_v c_v \bar{x}_v \leq 2 \sum_v c_v x_v^* = 2LP^* \leq 2 \text{ OPT}.$$

□

Worked example 11.4 (A half-integral triangle). In an unweighted triangle, $x_v = 1/2$ for all three vertices is LP-feasible with value $3/2$, while every integral cover has size 2. Threshold rounding selects all three vertices, attaining ratio $3/2$ on this example and respecting the worst-case bound two.

The LP method is polynomial because linear programming is polynomial-time solvable. The next unit derives a direct linear-time algorithm that follows the dual without invoking a general LP solver.

The Primal-Dual Method for Weighted Vertex Cover

EXAMINATION SYLLABUS: ITEM 12

12.1 Primal and dual relaxations

The fractional weighted vertex-cover LP and its dual are

$$\begin{array}{ll}
 \text{Primal: } \min \sum_v c_v x_v & \text{Dual: } \max \sum_e y_e \\
 x_u + x_v \geq 1 \quad (uv \in E) & \sum_{e \ni v} y_e \leq c_v \quad (v \in V) \\
 x_v \geq 0 & y_e \geq 0.
 \end{array}$$

Interpret y_e as money contributed by edge e toward buying its endpoints. A vertex is **tight** when $\sum_{e \ni v} y_e = c_v$.

12.2 Algorithm

Maintain residual slack $r_v = c_v - \sum_{e \ni v} y_e \geq 0$. On an uncovered edge uv , raise y_{uv} until one endpoint becomes tight, and select every endpoint that becomes tight.

Listing 12.1: Primal-dual weighted vertex cover

```

1 function PRIMAL-DUAL-COVER(G, cost):
2   S = empty set
3   y[e] = 0 for every edge e
4   residual[v] = cost[v] for every vertex v
5   for each edge e = (u, v):
6     if u in S or v in S: continue
7     delta = min(residual[u], residual[v])
8     y[e] += delta
9     residual[u] -= delta
10    residual[v] -= delta
11    if residual[u] == 0: add u to S
12    if residual[v] == 0: add v to S
13  return S

```

Lemma 12.1 (Maintained invariants). *Throughout the algorithm, y is dual feasible, and every selected vertex is tight.*

Proof. Initially all dual variables are zero. Raising y_{uv} by the smaller endpoint slack preserves both endpoint constraints and affects no other vertex constraint. At least one new slack becomes zero. A vertex is selected only when its slack is zero and remains tight because later edges incident to an already selected vertex are skipped. $\square$

Theorem 12.2 (Primal-dual approximation). *The algorithm returns a vertex cover S with $c(S) \leq 2 \text{OPT}$ in $\mathcal{O}(|V| + |E|)$ time.*

Proof. Whenever an uncovered edge is processed, at least one endpoint becomes tight and is selected; hence that edge becomes covered. Skipped edges were already covered, so the final S is a vertex cover.

Every selected vertex is tight, so

$$c(S) = \sum_{v \in S} \sum_{e \ni v} y_e = \sum_{e=uv} y_e (\mathbf{1}[u \in S] + \mathbf{1}[v \in S]) \leq 2 \sum_e y_e.$$

Weak duality gives $\sum_e y_e \leq LP^* \leq \text{OPT}$. Therefore $c(S) \leq 2 \text{OPT}$. The arrays and one scan of vertices and edges take linear time. $\square$

Worked example 12.3 (A weighted triangle). Let the vertex costs be $c_a = 2, c_b = 3, c_c = 3$. Processing ab raises its dual by 2, makes a tight, and selects a . Edge ac is now covered. Processing bc raises its dual by 1, makes b tight, and selects b . The returned cover has cost 5, which is optimal here. The dual value is 3, and the general proof gives the certificate $5 \leq 2 \cdot 3$.

Remark 12.4. The method enforces exact complementary slackness on selected primal variables and a relaxed factor-two version on positive dual variables: an edge can have at most two selected endpoints. This controlled violation is the origin of the approximation factor.

Metric TSP: Double-Tree and Christofides-Serdyukov

EXAMINATION SYLLABUS: ITEM 13

13.1 Metric travelling salesperson problem

In metric TSP, a complete undirected graph has nonnegative weights satisfying

$$w(x, z) \leq w(x, y) + w(y, z).$$

The goal is a minimum-weight Hamiltonian cycle. The triangle inequality makes shortcutting safe.

Lemma 13.1 (Shortcutting). *If a closed walk visits every vertex, repeatedly bypassing repeated internal vertices produces a Hamiltonian cycle of no greater weight.*

Proof. Replacing consecutive edges $x \rightarrow y$ and $y \rightarrow z$ by $x \rightarrow z$ does not increase weight by the triangle inequality. Remove repeated occurrences until each vertex occurs once, retaining the return to the start. $\square$

Lemma 13.2 (MST lower bound). *If T is a minimum spanning tree and C^* an optimal TSP tour, then $w(T) \leq w(C^*)$.*

Proof. Deleting any edge from C^* leaves a spanning tree. Since T is a minimum spanning tree, it is no heavier than that tree, which is itself no heavier than C^* . $\square$

13.2 The double-tree algorithm

Compute an MST T , duplicate every edge, find an Euler tour of the resulting multigraph, and shortcut repeated vertices.

Theorem 13.3 (Double-tree approximation). *The double-tree algorithm is a polynomial-time 2-approximation for metric TSP.*

Proof. Duplicating T makes every degree even, so an Euler tour exists and has weight $2w(T)$. Shortcutting yields a Hamiltonian cycle C with

$$w(C) \leq 2w(T) \leq 2w(C^*).$$

MST computation, Euler traversal, and shortcutting are polynomial-time. $\square$

13.3 The Christofides-Serdyukov algorithm

Let O be the vertices of odd degree in the MST. By the handshaking lemma, $|O|$ is even. Compute a minimum-weight perfect matching M on the metric induced by O . The multigraph $T \cup M$ is connected and all its degrees are even. Take an Euler tour and shortcut it.

Listing 13.1: Christofides-Serdyukov algorithm

```

1 function CHRISTOFIDES(G, w):
2   T = minimum spanning tree of G
3   O = vertices of odd degree in T
4   M = minimum-weight perfect matching in G[O]
5   W = an Euler tour of the multigraph T union M
6   return Hamiltonian cycle obtained by shortcutting W

```

Lemma 13.4 (Matching bound). *The matching selected by the algorithm satisfies $w(M) \leq \frac{1}{2}w(C^*)$.*

Proof. List the vertices of O in the order in which they occur on the optimal tour. Shortcut between consecutive listed vertices. The result is a cycle on O of weight at most $w(C^*)$. Because $|O|$ is even, alternating edges of this cycle form two perfect matchings. The cheaper has weight at most half the cycle weight. Since M is a minimum-weight perfect matching on O , it is no heavier than that cheaper matching. $\square$

Theorem 13.5 (Christofides-Serdyukov approximation). *The algorithm returns a tour of weight at most $\frac{3}{2}$ OPT.*

Proof. The Eulerian multigraph has weight $w(T) + w(M)$. By the MST and matching bounds and by safe shortcutting,

$$w(C) \leq w(T) + w(M) \leq w(C^*) + \frac{1}{2}w(C^*) = \frac{3}{2}w(C^*).$$

Minimum-weight perfect matching in a general graph is polynomial-time computable, so the whole algorithm is polynomial. $\square$

Worked example 13.6 (Four points on a square). For the four Euclidean corners of a unit square, an MST has weight 3, its odd-degree set consists of two vertices, and their minimum matching has weight 1. The Eulerian multigraph has weight 4; shortcutting returns the optimal perimeter tour of weight 4.

Common pitfall 13.7. Without the triangle inequality, shortcutting can increase the cost by an arbitrary factor. The two guarantees are therefore guarantees for *metric* TSP, not unrestricted TSP.

Part V

Randomized Algebra and Hashing

Freivalds' Matrix-Product Verification

EXAMINATION SYLLABUS: ITEM 14

14.1 Verification versus multiplication

Given $A, B, C \in R^{n \times n}$, where R is a ring, the task is to decide whether $AB = C$. Recomputing AB by the classical algorithm costs $\Theta(n^3)$ ring operations. Freivalds' algorithm verifies the claimed product in $\Theta(n^2)$ operations with one-sided error.

Choose r uniformly from $\{0, 1\}^n$, compute

$$u = Br, \quad v = Au, \quad w = Cr,$$

and accept exactly when $v = w$.

Listing 14.1: One iteration of Freivalds' algorithm

```

1 function FREIVALDS(A, B, C):
2   choose r uniformly from {0,1}^n
3   return A * (B * r) == C * r

```

Lemma 14.1 (Random binary dot product). *If $x, y \in R^n$ and $x \neq y$, then for uniform $r \in \{0, 1\}^n$,*

$$\mathbb{P}(x^T r = y^T r) \leq \frac{1}{2}.$$

Proof. Choose j with $x_j \neq y_j$ and condition on all bits except r_j . If equality held for both $r_j = 0$ and $r_j = 1$, subtracting the two equalities would give $x_j = y_j$. Thus at most one of the two equally likely values of r_j can cause equality. $\square$

Theorem 14.2 (Freivalds). *One iteration has the following guarantees:*

- if $AB = C$, it accepts with probability 1;
- if $AB \neq C$, it rejects with probability at least $1/2$.

It uses $\Theta(n^2)$ ring operations and $\Theta(n)$ auxiliary storage.

Proof. The first statement is immediate. Otherwise $D = AB - C$ has a nonzero row d^T . Acceptance implies $Dr = 0$, and hence $d^T r = 0$. Apply the lemma to d and the zero vector. Three matrix-vector products cost $\Theta(n^2)$, while no matrix-matrix product is formed. $\square$

Repeating independently k times and accepting only if every iteration accepts reduces false acceptance to at most 2^{-k} , with time $\Theta(kn^2)$.

Worked example 14.3 (A corrupted entry). If $C = AB + E_{ij}$, then $Dr = -E_{ij}r$. The only potentially nonzero coordinate is $-r_j$, so the error is detected exactly when $r_j = 1$, with probability $1/2$. This shows the one-iteration bound is tight.

Common pitfall 14.4. The random vector must be chosen after the matrices are fixed. If an adversary sees r first, it can construct a nonzero error matrix D with $Dr = 0$.

Polynomial Identity Testing and the Schwartz-Zippel Lemma

EXAMINATION SYLLABUS: ITEM 15

15.1 Identity testing

An arithmetic circuit may describe a polynomial with exponentially many monomials, so expanding all coefficients can be infeasible. Polynomial identity testing asks whether two circuits compute the same formal polynomial; subtracting them reduces the problem to deciding whether $p \equiv 0$.

Over a finite field, equality as formal polynomials is stronger than equality as functions. For example $x^q - x$ vanishes on $\mathbb{F}_q$ but is not the zero polynomial. Random evaluation is therefore analyzed as a test of formal identity, with a sampling set large relative to the degree.

Theorem 15.1 (Schwartz-Zippel). *Let $p \in \mathbb{F}[x_1, \dots, x_n]$ be nonzero with total degree at most d , and let $S \subseteq \mathbb{F}$ be finite. If $r_1, \dots, r_n$ are independent and uniform in S , then*

$$\mathbb{P}(p(r_1, \dots, r_n) = 0) \leq \frac{d}{|S|}.$$

Proof. Induct on n . For one variable, a nonzero polynomial of degree at most d has at most d roots. For the induction step, write the polynomial in x_1 :

$$p(x_1, \dots, x_n) = \sum_{i=0}^k x_1^i q_i(x_2, \dots, x_n), \quad q_k \neq 0.$$

Then $\deg q_k \leq d - k$. By induction,

$$\mathbb{P}(q_k(r_2, \dots, r_n) = 0) \leq \frac{d - k}{|S|}.$$

Conditioned on $q_k(r_2, \dots, r_n) \neq 0$, the resulting univariate polynomial in x_1 is nonzero of degree k , so it vanishes at uniform r_1 with probability at most $k/|S|$. A union bound gives $(d - k + k)/|S| = d/|S|$. $\square$

Listing 15.1: Randomized polynomial identity test

```

1 function IDENTITY-TEST(circuit p, degree d, set S):
2   choose independent r[1], ..., r[n] uniformly from S
3   if EVALUATE(p, r) != 0: return NONZERO
4   return PROBABLY-ZERO

```

The test has one-sided error: a zero polynomial always evaluates to zero; a nonzero polynomial is falsely accepted with probability at most $d/|S|$. Independent repetition raises this probability to the corresponding power.

Worked example 15.2 (A zero set can be infinite). Over $\mathbb{R}$, $p(x, y) = x$ vanishes on the whole line $x = 0$. Nevertheless, for $S = \{0, 1\}$, a uniform point of S^2 is a root with probability $1/2 = d/|S|$. The lemma counts density in the sampled grid, not geometric size in the ambient field.

Common pitfall 15.3. The degree in the bound is total degree, and the sampled coordinates must be independent. Choosing $r_1 = \dots = r_n$ can place every sample on a special algebraic curve and invalidate the guarantee.

Universal Hashing and Multiply-Add

EXAMINATION SYLLABUS: ITEM 16

16.1 Universal families

Let U be a large key universe and $M = \{0, \dots, m-1\}$. A family $\mathcal{H} \subseteq M^U$ is ε -**universal** if, for every distinct $x, y \in U$,

$$\mathbb{P}_{h \sim \mathcal{H}}(h(x) = h(y)) \leq \varepsilon.$$

It is **universal** when $\varepsilon = 1/m$. The keys may be adversarial; the probability is over a hash function chosen after the key set is fixed.

Universality controls pairwise collisions without requiring a truly random function table of size $|U|$.

16.2 The multiply-add family

Identify U with a subset of $\mathbb{F}_p$, where $p \geq |U|$ is prime. Choose $a \in \mathbb{F}_p^*$ and $b \in \mathbb{F}_p$ independently and uniformly, and define

$$h_{a,b}(x) = ((ax + b) \bmod p) \bmod m.$$

Lemma 16.1 (Pair permutation). *For fixed $x \neq y$, the map*

$$(a, b) \mapsto (ax + b, ay + b)$$

is a bijection from $\mathbb{F}_p^ \times \mathbb{F}_p$ to the ordered pairs $(r, s) \in \mathbb{F}_p^2$ with $r \neq s$.*

Proof. Given $r \neq s$, the unique solution is

$$a = (r - s)(x - y)^{-1}, \quad b = r - ax.$$

It has $a \neq 0$. Conversely every $a \neq 0$ maps distinct x, y to distinct field elements. □

Theorem 16.2 (Universality of multiply-add). *The family $\{h_{a,b} : a \neq 0\}$ is universal.*

Proof. By the pair-permutation lemma, the two pre-reduction values are a uniform ordered pair $r \neq s$ in $\mathbb{F}_p$. For fixed r , at most $\lfloor (p-1)/m \rfloor$ choices of $s \neq r$ satisfy $s \equiv r \pmod{m}$. Therefore

$$\mathbb{P}(h(x) = h(y)) \leq \frac{\lfloor (p-1)/m \rfloor}{p-1} \leq \frac{1}{m}.$$

□

Worked example 16.3. Take $p = 11, m = 5, a = 3, b = 1$. Then $h(2) = 7 \bmod 5 = 2$ and $h(7) = 0$. A different random pair (a, b) produces a different deterministic function; universality is a statement about the random family, not about every member.

For a fixed set of n keys, the expected number of colliding unordered pairs is at most $\binom{n}{2}/m$. This simple consequence drives the perfect-hashing analysis later.

Multiply-Shift Hashing

EXAMINATION SYLLABUS: ITEM 17

17.1 Word-RAM formulation

Let the universe be $U = \{0, \dots, 2^w - 1\}$ and the range have $m = 2^\ell$ values, $\ell \leq w$. Choose a uniform odd w -bit integer a and set

$$h_a(x) = \left\lfloor \frac{(ax) \bmod 2^w}{2^{w-\ell}} \right\rfloor.$$

In machine language, multiply with w -bit overflow and return the top ℓ bits. Unlike multiply-add, no prime modulus or division is required.

Lemma 17.1 (Subtraction-window lemma). *If the top ℓ bits of two w -bit words X, Y are equal, then the top ℓ bits of $(Y - X) \bmod 2^w$ are either all zero or all one.*

Proof. Subtract the low $w - \ell$ bits first. The borrow into the high window is either zero or one. Since the high windows of X and Y are equal, their difference is all zeros if there is no borrow and all ones in two's-complement arithmetic if there is a borrow. $\square$

Theorem 17.2 (Multiply-shift universality bound). *For any distinct $x, y \in U$,*

$$\mathbb{P}_{a \text{ odd}}(h_a(x) = h_a(y)) \leq \frac{2}{m}.$$

Proof. Assume $x < y$, write $y - x = 2^r q$ with q odd and $0 \leq r < w$. Multiplication by odd q permutes the odd residues modulo 2^w . Hence for uniform odd a , $z = aq \bmod 2^w$ is a uniform odd word: its least significant bit is 1 and all other bits are uniform.

If a collision occurs, the subtraction-window lemma says the top ℓ bits of

$$a(y - x) = 2^r z \pmod{2^w}$$

are all zero or all one. We bound this event by cases.

If $r < w - \ell$, the high window consists of ℓ uniform bits of z , so the two constant patterns have total probability $2/2^\ell$. If $r = w - \ell$, one bit in the window is the fixed low bit of z ; one constant pattern is impossible and the other has probability $2/2^\ell$. If $r > w - \ell$, the window contains both a forced zero from the shift and the forced one from z , so neither constant pattern is possible. In all cases the collision probability is at most $2/m$. $\square$

Remark 17.3. The factor two is the price for using only multiplication and a shift. The family is especially attractive on the word RAM because reduction modulo 2^w is automatic unsigned overflow.

Common pitfall 17.4. The multiplier must be odd. An even multiplier discards low-order information and need not permute odd residues, breaking the proof.

Perfect Hashing

EXAMINATION SYLLABUS: ITEM 18

18.1 One-level perfect hashing

Let $S \subseteq U$ be a fixed static set of s keys. A hash function is **perfect for** S if it is injective on S . Choose a universal function into $m = s^2$ cells. By the union bound,

$$\mathbb{P}(h \text{ has a collision on } S) \leq \binom{s}{2} \frac{1}{s^2} < \frac{1}{2}.$$

Thus repeated independent choice finds an injective function in fewer than two trials on average. This gives constant-time lookup but quadratic space.

18.2 Two-level FKS hashing

Choose a first-level universal hash into s buckets. Let S_i be bucket i and $s_i = |S_i|$. Bucket i receives a second-level table of size $m_i = s_i^2$, together with an independently chosen universal hash function that is injective on S_i .

Lemma 18.1 (Expected second-level space). *For a universal first-level hash into s buckets,*

$$\mathbb{E} \left[\sum_i s_i^2 \right] \leq 2s - 1.$$

Proof. Let X be the number of colliding unordered key pairs. Since

$$\sum_i s_i^2 = s + 2 \sum_i \binom{s_i}{2} = s + 2X$$

and every pair collides with probability at most $1/s$,

$$\mathbb{E}X \leq \binom{s}{2} \frac{1}{s} = \frac{s-1}{2}.$$

Substitution proves the claim. □

By Markov's inequality, $\mathbb{P}(\sum_i s_i^2 > 4s) < 1/2$. Choose first-level functions until the total is at most $4s$, then choose an injective second-level function independently in each bucket.

Theorem 18.2 (Static perfect hashing). *The two-level construction uses $\mathcal{O}(s)$ space, answers every membership query in worst-case $\mathcal{O}(1)$ time, and has expected $\mathcal{O}(s)$ construction time.*

Proof. The accepted first level allocates at most $4s$ second-level cells. A query computes one first-level and one second-level hash, then compares the stored key. The first-level acceptance probability exceeds $1/2$; in a bucket of size s_i , a random universal function into s_i^2 cells is injective with probability exceeding $1/2$. Hence each selection needs fewer than two trials in expectation. Testing trials costs linear time in the keys involved, and the sums over buckets are $\mathcal{O}(s)$. $\square$

Worked example 18.3 (Space calculation). If the first-level bucket sizes are 3, 1, 0, 2, then $s = 6$ and second-level space is $3^2 + 1^2 + 0^2 + 2^2 = 14 < 4s = 24$. Each nonempty bucket now receives its own independently selected collision-free table.

Collision Resolution: Chaining and Linear Probing

EXAMINATION SYLLABUS: ITEM 19

19.1 Separate chaining

With n stored keys and m buckets, the load factor is $\alpha = n/m$. Separate chaining stores all keys of hash value i in a list for bucket i . Insertion at the front is constant time; search scans one list.

Theorem 19.1 (Expected chaining time). *If h is chosen from a universal family, a search for any fixed key takes expected $\mathcal{O}(1 + \alpha)$ time.*

Proof. For a key not in the table, the expected number of stored keys colliding with it is at most $n/m = \alpha$ by linearity of expectation. For a stored key, add one for the key itself and apply the same bound to the other keys. List scanning time is proportional to this bucket length. $\square$

19.2 Linear probing

Open addressing stores keys directly in a table and requires $\alpha < 1$. Linear probing examines

$$h(x), h(x) + 1, h(x) + 2, \dots \pmod{m}$$

until the key or an empty cell is found. It has excellent locality but forms clusters. The following intentionally simplified analysis assumes that all home addresses are independent and uniform, a stronger model than ordinary pairwise universality.

Lemma 19.2 (An interval is unlikely to be overfull). *Assume $n \leq m/2$. For a fixed circular interval I of length r , let X_I be the number of stored keys hashing into I . There exists a constant $\beta < 1$ such that*

$$\mathbb{P}(X_I \geq r) \leq \beta^r.$$

Proof. Here X_I is binomial with mean $\mu = nr/m \leq r/2$. The multiplicative Chernoff bound at twice the mean gives

$$\mathbb{P}(X_I \geq r) \leq \left(\frac{e}{4}\right)^{r/2}.$$

Take $\beta = \sqrt{e/4} < 1$. A smaller mean only decreases the upper tail. $\square$

Theorem 19.3 (Simplified linear-probing bound). *Under independent uniform hashing and $\alpha \leq 1/2$, successful and unsuccessful linear-probing searches use expected $\mathcal{O}(1)$ probes.*

Proof. Every maximal occupied block has an empty predecessor. Therefore every key stored in that block has its home address inside the block; otherwise it would have stopped at the empty predecessor. If the probe sequence starting at a fixed home position crosses a block of length L , some length- L interval containing that position receives at least L home addresses. There are at most L such intervals. Hence, by the lemma and a union bound,

$$\mathbb{P}(T \geq r) \leq \sum_{L \geq r} L\beta^L.$$

Summing the tail probabilities over r gives a convergent geometric-power series, so $\mathbb{E}T = \mathcal{O}(1)$. A successful search stops no later than an unsuccessful search started from the same home address, up to one final comparison. $\square$

Common pitfall 19.4. Pairwise universality alone is insufficient for this proof; linear probing needs stronger randomness because whole intervals, not just key pairs, matter. Deletion must also use tombstones or backward-shift repair: simply clearing a cell can make a later key unreachable.

Binary Probing

EXAMINATION SYLLABUS: ITEM 20

20.1 Probe order and dyadic blocks

Let $m = 2^\ell$. Binary probing examines

$$h(x) \oplus 0, h(x) \oplus 1, h(x) \oplus 2, \dots,$$

where $\oplus$ is bitwise XOR. The first 2^k positions are exactly the dyadic block $B_k(h(x))$ of length 2^k sharing the same high $\ell - k$ bits as $h(x)$. Thus the search expands through nested blocks of sizes $1, 2, 4, \dots$

Call a dyadic interval I **demanded** if at least $|I|$ stored keys have home addresses in I .

Lemma 20.1 (Full-block witness). *If a dyadic block $B_k(h(x))$ is full when a key is probed, then either that block or a sibling of one of its ancestors is demanded.*

Proof. If the full block is not demanded, some key occupying it must have arrived from outside. Follow the nested probe blocks of the earliest such arriving key. Immediately before it enters the block, its own current dyadic block is full. That block is a sibling of an ancestor of $B_k(h(x))$. If it is not demanded, repeat the argument. Moving to a larger ancestor must eventually stop because the table is not full. The first witness that cannot be explained by earlier overflow is demanded. $\square$

20.2 Simplified expected-time analysis

Assume independent uniform home addresses and load $\alpha = n/m \leq 1/4$. For a fixed interval I of length r , a union bound over choices of r keys gives

$$\mathbb{P}(I \text{ is demanded}) \leq \binom{n}{r} \left(\frac{r}{m}\right)^r \leq (e\alpha)^r.$$

Put $\rho = e\alpha < 1$. The possible witnesses for a full level- k block have lengths $2^k, 2^{k+1}, \dots$, one sibling per higher level. Therefore

$$\mathbb{P}(B_k(h(x)) \text{ is full}) \leq \sum_{j \geq k} \rho^{2^j} = \mathcal{O}(\rho^{2^k}).$$

Theorem 20.2 (Binary-probing bound). *Under the simplified random-hashing model and $\alpha \leq 1/4$, insertion and search take expected $\mathcal{O}(1)$ probes.*

Proof. If more than 2^k positions are probed, the level- k block was full. Group possible running times by powers of two:

$$\mathbb{E}T \leq 1 + \sum_{k \geq 0} 2^{k+1} \mathbb{P}(T > 2^k) \leq 1 + \sum_{k \geq 0} 2^{k+1} \mathcal{O}(\rho^{2^k}) = \mathcal{O}(1).$$

The last series converges because its probability term decreases doubly exponentially in k . □

Worked example 20.3 (Eight-cell probe order). For $m = 8$ and home address $h = 5 = 101_2$, the order is

$$5, 4, 7, 6, 1, 0, 3, 2.$$

The first two positions form the dyadic block $10*_2$, the first four form $1*_2$, and all eight form the root block.

Remark 20.4. Binary probing is not the same as quadratic probing. Its XOR order is chosen so that prefixes of the probe sequence are standard dyadic intervals, which is exactly the hierarchy used in the analysis.

Part VI

Satisfiability and Primalty

SAT, k -SAT, and Deterministic 2-SAT

EXAMINATION SYLLABUS: ITEM 21

21.1 Boolean satisfiability

A **literal** is a variable x_i or its negation $\neg x_i$. A **clause** is a disjunction of literals, and a formula is in conjunctive normal form (CNF) if it is a conjunction of clauses. In k -CNF every clause has at most k literals; k -SAT asks whether a satisfying assignment exists.

General SAT and 3-SAT are NP-complete. Two-literal clauses have additional implication structure:

$$(P \vee Q) \equiv (\neg P \Rightarrow Q) \equiv (\neg Q \Rightarrow P).$$

21.2 The implication graph

For a 2-CNF formula F , create one vertex for each literal $x_i, \neg x_i$. For every clause $P \vee Q$, add edges

$$\neg P \rightarrow Q, \quad \neg Q \rightarrow P.$$

The graph has $2n$ vertices and $2m$ edges. An edge means that every satisfying assignment that makes its tail true must make its head true.

Theorem 21.1 (2-SAT criterion). *A 2-CNF formula is satisfiable if and only if no variable x_i and its negation $\neg x_i$ belong to the same strongly connected component of the implication graph.*

Proof. If x_i and $\neg x_i$ are in one SCC, there are implication paths in both directions. Any satisfying assignment that makes x_i true must therefore make $\neg x_i$ true; if it makes x_i false, then $\neg x_i$ is true and the reverse path forces x_i true. Both cases are impossible.

Conversely, contract SCCs to obtain a DAG and let $r(C)$ be a topological rank that strictly increases along every condensation edge. Set a literal L true exactly when

$$r(C(L)) > r(C(\neg L)).$$

Exactly one of $L, \neg L$ is true because their SCCs differ.

Suppose a clause $P \vee Q$ were false. Then

$$r(C(P)) < r(C(\neg P)), \quad r(C(Q)) < r(C(\neg Q)).$$

The implication edges $\neg P \rightarrow Q$ and $\neg Q \rightarrow P$ give

$$r(C(\neg P)) \leq r(C(Q)), \quad r(C(\neg Q)) \leq r(C(P)).$$

Combining the four inequalities yields a strict cycle

$$r(C(P)) < r(C(\neg P)) \leq r(C(Q)) < r(C(\neg Q)) \leq r(C(P)),$$

a contradiction. Hence every clause is satisfied. □

Listing 21.1: Deterministic 2-SAT

```

1 function TWO-SAT(F):
2     build implication graph G
3     compute SCC identifier comp[L] for every literal L
4     for each variable x:
5         if comp[x] == comp[not x]: return UNSAT
6     topologically rank the SCC condensation
7     value[x] = (rank(comp[x]) > rank(comp[not x]))
8     return value

```

Kosaraju, Tarjan, or Gabow SCC computation runs in $\mathcal{O}(n + m)$ time and space, so 2-SAT is linear-time.

Worked example 21.2 (An immediate contradiction). The formula $(x \vee x) \wedge (\neg x \vee \neg x)$ adds implications $\neg x \rightarrow x$ and $x \rightarrow \neg x$. The two literals form one SCC, certifying unsatisfiability.

Common pitfall 21.3. Testing SCC separation is sufficient only because the formula is 2-CNF. A three-literal clause cannot be represented by two single-literal implications with the same semantics.

A Random-Walk Algorithm for 2-SAT

EXAMINATION SYLLABUS: ITEM 22

22.1 Algorithm

Start from any assignment. While the formula is false, choose any false clause and flip one of its two variables uniformly at random. Stop after a fixed budget if no satisfying assignment has appeared.

Listing 22.1: Random-walk 2-SAT

```

1 function RANDOM-WALK-2SAT(F, n):
2   choose any assignment x
3   for step = 1 to 2*n*n:
4     if x satisfies F: return x
5     choose any false clause (L1 or L2)
6     choose one of its two variables uniformly and flip it
7   return PROBABLY-UNSAT

```

This is a one-sided algorithm: a returned assignment is always a valid certificate, while failure to find one may be erroneous.

22.2 Hamming-distance analysis

Assume F is satisfiable and fix one satisfying assignment x^* . Let D_t be the Hamming distance from the current assignment to x^* . In a false clause, the current assignment falsifies both literals, while x^* satisfies at least one. Therefore at least one of the two variables differs from x^* , and a random flip decreases D_t with probability at least $1/2$. At distance n , every variable differs, so the next flip necessarily decreases the distance.

Consider the slower comparison walk on $\{0, 1, \dots, n\}$ that moves left or right with equal probability internally, reflects from n , and is absorbed at 0. Let E_i be its expected absorption time from i . Then

$$E_0 = 0, \quad E_i = 1 + \frac{E_{i-1} + E_{i+1}}{2} \quad (0 < i < n), \quad E_n = 1 + E_{n-1}.$$

Lemma 22.1 (Expected hitting time). *The solution is*

$$E_i = 2ni - i^2 \leq n^2.$$

Proof. Direct substitution verifies the interior recurrence and both boundary conditions. The recurrence system has a unique solution because it is the finite system of first-step equations for an absorbing Markov chain. $\square$

The actual distance process is at least as likely to move toward zero, so its expected time to reach x^* is at most n^2 .

Theorem 22.2 (Randomized 2-SAT guarantee). *If F is satisfiable, the algorithm finds a satisfying assignment within $2n^2$ flips with probability at least $1/2$. If F is unsatisfiable, it never reports a false assignment as satisfying.*

Proof. Let T be the hitting time of x^* . We have $\mathbb{E}T \leq n^2$, so Markov's inequality gives $\mathbb{P}(T > 2n^2) \leq 1/2$. The one-sided statement follows because the formula is explicitly checked before an assignment is returned. $\square$

With a naive $\mathcal{O}(m)$ scan to find a false clause after every flip, the time is $\mathcal{O}(mn^2)$. Independent restarts reduce the failure probability geometrically.

Schoning's Randomized Algorithm for 3-SAT

EXAMINATION SYLLABUS: ITEM 23

23.1 Algorithm and local progress

One trial begins with a uniform random assignment. For at most $3n$ steps, choose any false three-literal clause and flip one of its three variables uniformly. Repeat independent trials.

Fix a satisfying assignment x^* . A false clause contains at least one variable on which the current assignment differs from x^* , so a flip moves toward x^* with probability at least $1/3$. Unlike 2-SAT, this drift is away from the target in the worst case, which makes restarts essential.

23.2 Success probability of one trial

Lemma 23.1 (Truncated biased walk). *If the initial distance from x^* is i , the probability of reaching x^* within $3n$ flips is*

$$\Omega\left(\frac{2^{-i}}{\sqrt{n}}\right).$$

Proof. Couple the distance with the worst walk that moves one step toward zero with probability $1/3$ and away with probability $2/3$. A more favorable move probability can only increase the chance of hitting zero.

Starting at i , consider the first $3i \leq 3n$ moves. If exactly $2i$ moves are toward zero and i are away, the net displacement is i toward zero. The walk either reaches zero earlier or ends there. The event has probability

$$\binom{3i}{2i} \left(\frac{1}{3}\right)^{2i} \left(\frac{2}{3}\right)^i = \Omega\left(\frac{2^{-i}}{\sqrt{i+1}}\right) = \Omega\left(\frac{2^{-i}}{\sqrt{n}}\right),$$

where the middle estimate follows from Stirling's formula. □

The initial distance I is binomial $\text{Bin}(n, 1/2)$. Averaging the lemma gives

$$\mathbb{P}(\text{one trial succeeds}) \geq \Omega\left(\frac{1}{\sqrt{n}} \sum_{i=0}^n \binom{n}{i} 2^{-n} 2^{-i}\right) = \Omega\left(\frac{1}{\sqrt{n}} \left(\frac{3}{4}\right)^n\right).$$

Theorem 23.2 (Schoning's 3-SAT bound). *After $\mathcal{O}(\sqrt{n}(4/3)^n)$ independent trials, a satisfiable 3-CNF is solved with constant probability. The running time is $\mathcal{O}^*((4/3)^n)$, where $\mathcal{O}^*$ suppresses polynomial factors. The error is one-sided.*

Proof. One trial succeeds with probability $p = \Omega(n^{-1/2}(3/4)^n)$. With $R = \Theta(1/p)$ independent trials, failure is at most $(1 - p)^R \leq e^{-pR}$, a constant below one. Every reported assignment is checked, so an unsatisfiable formula is never falsely accepted. $\square$

Listing 23.1: Schoning's 3-SAT algorithm

```
1 repeat R = Theta(sqrt(n) * (4/3)^n) times:
2     choose x uniformly from {0,1}^n
3     for step = 0 to 3*n:
4         if x satisfies F: return x
5         choose any false clause
6         choose one of its three variables uniformly and flip it
7 return PROBABLY-UNSAT
```

Remark 23.3. The easy event “all necessary corrections occur immediately” gives only a $(3/2)^n$ bound. The sharper $4/3$ base comes from crediting paths that temporarily move away from a satisfying assignment and later return.

Miller-Rabin Primality Testing

EXAMINATION SYLLABUS: ITEM 24

24.1 From Fermat to strong witnesses

For prime N and $a \not\equiv 0 \pmod{N}$, Fermat's theorem gives $a^{N-1} \equiv 1 \pmod{N}$. The converse fails, even for every coprime base on Carmichael numbers. Miller-Rabin also inspects the chain of repeated square roots leading to a^{N-1} .

For an odd $N > 3$, write

$$N - 1 = 2^s d, \quad d \text{ odd.}$$

For a chosen base a , compute $x = a^d \bmod N$, followed by repeated squaring.

Listing 24.1: One Miller-Rabin round

```

1 function MILLER-RABIN-ROUND(N, a):
2     if gcd(a, N) > 1: return COMPOSITE
3     write N - 1 = 2^s * d with d odd
4     x = pow_mod(a, d, N)
5     if x == 1 or x == N - 1: return PASS
6     for r = 1 to s - 1:
7         x = x * x mod N
8         if x == N - 1: return PASS
9         if x == 1: return COMPOSITE
10    return COMPOSITE

```

Lemma 24.1 (Square roots of one modulo a prime). *If N is an odd prime and $x^2 \equiv 1 \pmod{N}$, then $x \equiv \pm 1 \pmod{N}$.*

Proof. The congruence says $N \mid (x - 1)(x + 1)$. Euclid's lemma for a prime gives $N \mid x - 1$ or $N \mid x + 1$. □

For a prime, the squaring chain ends at 1. If it does not start at 1, the first occurrence of 1 must be preceded by -1 . Hence a prime passes every round.

24.2 Density of witnesses

Lemma 24.2 (At least half the bases witness compositeness). *For every odd composite N , a uniform base from $\{2, \dots, N - 1\}$ causes a Miller-Rabin round to report composite with probability at least $1/2$.*

Proof. Bases not coprime to N are detected by the gcd, so consider the group $G = (\mathbb{Z}/N\mathbb{Z})^*$.

If the Fermat test $a^{N-1} = 1$ fails for some unit, the passing units form the kernel of the homomorphism $a \mapsto a^{N-1}$. It is a proper subgroup of G , hence has at most $|G|/2$ elements; strong liars are a subset of it.

It remains to consider the Carmichael case in which every unit passes Fermat. Among exponents $2^j d$, choose the largest $j < s$ for which some unit has power -1 . The units whose $2^j d$ -th power is ± 1 form a subgroup H containing every strong liar. This subgroup is proper. Indeed, a Carmichael number has two coprime factors $N_1, N_2 > 1$. Choose $c^{2^j d} \equiv -1 \pmod{N}$, and use the Chinese remainder theorem to choose $y \equiv c \pmod{N_1}$, $y \equiv 1 \pmod{N_2}$. Then y is a unit, but its $2^j d$ -th power is -1 modulo one factor and 1 modulo the other, so it is neither 1 nor -1 modulo N . Thus $y \notin H$.

Every proper subgroup has size at most half the finite group, so in either case at most half the coprime bases are strong liars. Including nonunits only increases the detection probability. $\square$

Theorem 24.3 (Miller-Rabin guarantee). *One round always accepts a prime and falsely passes an odd composite with probability at most $1/2$. After k independent rounds the error is at most 2^{-k} .*

Proof. Prime correctness follows from Fermat's theorem and the square-root lemma. The composite bound is the witness-density lemma. Independent rounds multiply the false-pass probabilities. $\square$

24.3 Complexity and example

Let $n = \lceil \log_2 N \rceil$. Binary modular exponentiation uses $\mathcal{O}(n)$ modular multiplications. With schoolbook n -bit multiplication, one round costs $\mathcal{O}(n^3)$ bit operations and $\mathcal{O}(n)$ working bits beyond the input and arithmetic buffers.

For $N = 21$, $N - 1 = 4 \cdot 5$. With base 2, the chain begins

$$2^5 \equiv 11, \quad 11^2 \equiv 16, \quad 16^2 \equiv 4 \pmod{21}.$$

It reaches neither an initial 1 nor a $-1 \equiv 20$ before the final Fermat value, so the round correctly reports composite.

Common pitfall 24.4. Real implementations must handle $N < 4$, even N , repeated bases, and overflow in modular multiplication. The phrase “probably prime” describes the randomized evidence; it is not permission to skip these deterministic edge cases.

Part VII

Online Algorithms and Spanning Trees

Online Algorithms: Scooter Rental and Caching

EXAMINATION SYLLABUS: ITEM 25

25.1 Competitive analysis

An online algorithm receives requests sequentially and must act before future requests are known. The offline optimum $\text{OPT}(\sigma)$ knows the whole request sequence σ . For a minimization problem, an online algorithm is ρ -**competitive** if

$$\text{cost}_A(\sigma) \leq \rho \text{OPT}(\sigma) + \beta$$

for every sequence, where β is an input-independent additive constant. When $\beta = 0$, this is a strict competitive ratio.

The comparison is deliberately demanding: the online algorithm is measured against a clairvoyant solution on the same sequence.

25.2 The scooter-rental problem

Renting costs $r > 0$ per day; buying costs $p > 0$. The number of usage days is unknown. Let $q = \lceil p/r \rceil$. The threshold algorithm rents for at most $q - 1$ days and, if another day is needed, buys before day q .

Theorem 25.1 (Rental threshold). *The threshold algorithm is strictly less than 2-competitive. If $p/r = q \in \mathbb{N}$, its exact ratio is $2 - r/p = 2 - 1/q$, and no deterministic online algorithm has a smaller ratio.*

Proof. If the horizon $D < q$, then $Dr < p$; both the algorithm and optimum rent, so the ratio is one. If $D \geq q$, the algorithm pays

$$r(q - 1) + p < 2p,$$

while the optimum pays p . This proves the bound. When $p = qr$, the ratio in the second case is exactly $(r(q - 1) + p)/p = 2 - 1/q$.

For deterministic optimality, any algorithm has a fixed first buying day k as long as the adversary keeps requesting use. If $k \leq q$, stop on day k ; the ratio is

$$\frac{(k - 1)r + p}{kr} = \frac{k - 1 + q}{k} \geq 2 - \frac{1}{q}.$$

If $k > q$, continue through its buying day; the ratio is at least $((k - 1)r + p)/p \geq 2$. If it never buys, the ratio is unbounded. Hence no deterministic rule beats the threshold ratio. $\square$

25.3 The paging problem

A cache holds k pages. A request costs zero if its page is present. On a miss, the requested page is fetched at cost one and, if the cache is full, one resident page must be evicted. The offline optimum is Belady's farthest-in-the-future rule.

LRU evicts the least recently requested page and is k -competitive. The main randomized algorithm in the course improves the factor to logarithmic.

25.4 The randomized marking algorithm

Divide requests into phases, each a maximal sequence containing at most k distinct pages. At the start of a phase, clear all marks. Every requested page is marked. On a miss, evict a uniformly random unmarked page; marked pages are never evicted during the current phase.

Listing 25.1: Randomized marking algorithm

```

1 on request page x:
2   if x would be the (k+1)-st distinct page of this phase:
3     clear every mark and start a new phase
4   if x is not in cache:
5     evict a uniformly random unmarked cached page
6     fetch x
7   mark x

```

A page in phase i is **clean** if it was not requested in phase $i - 1$; otherwise it is **stale**. Let m_i be the number of clean pages in phase i .

Lemma 25.2 (Expected faults in one phase). *The marking algorithm incurs at most $m_i H_k$ expected faults in phase i , where $H_k = 1 + 1/2 + \dots + 1/k$.*

Proof. Every clean page faults at most once, contributing m_i . Consider first requests to stale pages. After s distinct stale pages have already been requested, $k - s$ stale candidates remain. At most m_i of them have been displaced by clean-page insertions. Symmetry of uniform eviction among unmarked pages therefore bounds the probability that the next stale page has been displaced by $m_i/(k - s)$ (capped at one). Summing over the possible values of s and adding clean faults gives at most

$$m_i + m_i \left(\frac{1}{k} + \frac{1}{k-1} + \dots + \frac{1}{2} \right) = m_i H_k.$$

□

Lemma 25.3 (Offline lower bound). *If q_i is the number of offline faults in phase i , then*

$$q_{i-1} + q_i \geq m_i$$

for $i > 1$.

Proof. Phase $i - 1$ contains k distinct pages unless it is the last incomplete phase. Phase i contains m_i clean pages not among those k . Across the two phases, $k + m_i$ distinct pages are requested. A cache of size k must fetch at least m_i of them during the two phases. □

Theorem 25.4 (Competitive ratio of marking). *The randomized marking algorithm is $2H_k = \mathcal{O}(\log k)$ -competitive in expectation, up to the initial-cache additive term.*

Proof. Sum the phase bound and use the offline lower bound:

$$\mathbb{E}[\text{cost}_A] \leq H_k \sum_i m_i \leq 2H_k \sum_i q_i + \mathcal{O}(kH_k) = 2H_k \text{OPT} + \mathcal{O}(kH_k).$$

The factor two arises because each offline fault may be charged by the phase before it and the phase after it. $\square$

Worked example 25.5 (Phases for $k = 3$). For requests $1, 2, 3, 1, 4, 2, 3, 4, 5, \dots$, the first phase is $1, 2, 3, 1$; the request 4 starts the second phase. Within a phase, once a page is requested it is marked and protected. Randomness is used only to choose among pages that have not yet appeared in the phase.

Common pitfall 25.6. The adversarial model matters. The standard randomized competitive guarantee assumes an oblivious adversary that fixes the request sequence independently of the algorithm's private random choices.

Minimum Spanning Trees: Boruvka, Prim, and Kruskal

EXAMINATION SYLLABUS: ITEM 26

26.1 Spanning trees and safe edges

Let $G = (V, E)$ be a connected undirected graph with real edge weights. A spanning tree connects all vertices with $|V| - 1$ edges and no cycle. A minimum spanning tree (MST) has minimum total weight.

For $S \subset V$, the cut $\delta(S)$ consists of edges with exactly one endpoint in S . An edge is **light** for the cut if it has minimum weight among the crossing edges.

Lemma 26.1 (Cut property). *Let F be a forest contained in some MST, and let S be the vertex set of one component of F . Any light edge $e \in \delta(S)$ is safe: $F \cup \{e\}$ is contained in some MST.*

Proof. Let T be an MST containing F . If $e \in T$, there is nothing to prove. Otherwise $T + e$ contains one cycle. This cycle has another edge $f \in \delta(S)$; moreover $f \notin F$, since no edge of the component forest crosses its own cut. Lightness gives $w(e) \leq w(f)$. Then $T' = T - f + e$ is a spanning tree with $w(T') \leq w(T)$, so it is an MST and contains $F + e$. $\square$

This exchange argument is the common correctness engine of all three algorithms. Ties may be broken by a fixed total order refining weight; this does not change which trees minimize the original weights.

26.2 Boruvka's algorithm

Start with singleton components. In each phase, every current component selects its lightest outgoing edge; add the selected safe edges and contract the new components.

Listing 26.1: Boruvka's algorithm

```

1 function BORUVKA(G):
2   F = empty forest
3   make one DSU set per vertex
4   while more than one component remains:
5     best[C] = NONE for every component C
6     for each edge (u, v):
7       Cu = FIND(u); Cv = FIND(v)
8       if Cu == Cv: continue
9       update best[Cu] and best[Cv] with this edge
10    for each component C with best[C] != NONE:
11      (u, v) = best[C]
12      if FIND(u) != FIND(v):
13        add (u, v) to F; UNION(u, v)
14  return F

```

Theorem 26.2 (Boruvka correctness and complexity). *Boruvka returns an MST in $\mathcal{O}(|E| \log |V|)$ time.*

Proof. Every chosen edge is light across a component cut and therefore safe by lemma 26.1. Process the selected edges in the fixed tie-breaking order and skip any that have already become internal; the invariant that the forest extends to an MST is preserved. Every old component with an outgoing edge merges with at least one other component, so the number of components at least halves. There are $\mathcal{O}(\log |V|)$ phases. Each phase scans all edges and performs near-linear DSU work, hence costs $\mathcal{O}(|E|)$ at this level of analysis. $\square$

Boruvka is naturally parallel: component minima can be found simultaneously.

26.3 Jarnik-Prim's algorithm

Prim grows one tree. For every outside vertex v , maintain the lightest edge from the current tree to v and store its weight as $\text{key}[v]$ in a priority queue.

Listing 26.2: Jarnik-Prim with a binary heap

```

1 function PRIM(G, root):
2   key[root] = 0; key[v] = infinity for v != root
3   parent[v] = NONE
4   Q = all vertices keyed by key
5   while Q is not empty:
6     u = EXTRACT-MIN(Q)
7     for each edge (u, v) with v still in Q:
8       if w(u, v) < key[v]:
9         parent[v] = u
10        DECREASE-KEY(Q, v, w(u, v))
11  return edges (parent[v], v) for v != root

```

Theorem 26.3 (Prim correctness and complexity). *Prim returns an MST. With adjacency lists and a binary heap it runs in $\mathcal{O}((|V| + |E|) \log |V|)$ time and $\mathcal{O}(|V| + |E|)$ space.*

Proof. Immediately before a vertex is extracted, its parent edge is the lightest edge crossing from the current tree to that vertex, and the minimum key is a light edge across the cut between the whole current tree and the remaining vertices. The cut property makes it safe. Induction proves that all selected edges extend to an MST; after $|V| - 1$ additions they form one. Each vertex is extracted once and each edge can cause at most one successful decrease-key per endpoint, yielding the heap bound. $\square$

26.4 Kruskal's algorithm

Kruskal globally sorts edges by nondecreasing weight and inserts an edge exactly when it joins two different current components.

Listing 26.3: Kruskal's algorithm

```

1 function KRUSKAL(G):
2     sort edges by nondecreasing (weight, tie-break id)
3     make one DSU set per vertex
4     F = empty set
5     for edge (u, v) in sorted order:
6         if FIND(u) != FIND(v):
7             add (u, v) to F
8             UNION(u, v)
9     return F

```

Theorem 26.4 (Kruskal correctness and complexity). *Kruskal returns an MST in*

$$\mathcal{O}(|E| \log |E| + |E|\alpha(|V|)) = \mathcal{O}(|E| \log |V|)$$

time with union by rank and path compression.

Proof. When a considered edge e joins different forest components, take the cut of one component. Every earlier edge crossing this cut has already been considered and would have joined the components, so it is now internal only if the components have already merged. Thus e is light among currently eligible crossing edges and is safe by the cut property. The forest invariant and the final edge count prove correctness. Sorting dominates the inverse-Ackermann DSU operations. $\square$

Worked example 26.5 (Three algorithms, one tree). Let the edges be $ab : 1, bc : 2, ac : 4, bd : 3, cd : 5$. Kruskal chooses (ab, bc, bd) . Prim from a extracts the same three boundary minima. Boruvka first chooses ab for a, b , bc for c , and bd for d , producing the same MST of weight 6 in one phase after duplicate choices are removed.

Corollary 26.6 (Uniqueness with distinct weights). *If all edge weights are distinct, the MST is unique.*

Proof. If two MSTs differed, take the lightest edge e in their symmetric difference and add it to the tree not containing it. On the created cycle, another differing edge f is heavier than e . Replacing f by e would produce a lighter spanning tree, a contradiction. $\square$

Negative edge weights cause no difficulty. If the graph is disconnected, the same algorithms return a minimum spanning forest rather than a spanning tree.

Examination Syllabus Coverage Index

This appendix is a completeness certificate for the supplied Spring 2026 examination programme. Every question 1, . . . , 26 appears exactly once as a framed marker in the main text. Each marked lecture unit contains the definitions, central algorithms, correctness arguments, complexity bounds, and worked explanations required for that question.

Item	Required topic	Page
1	Primitive roots; DFT and inverse DFT; linear, cyclic, and negacyclic convolution; Karatsuba and DFT polynomial multiplication	2
2	FFT problem; decimation in time and frequency; general and six-step FFT; complexity	6
3	Circuit model; comparator, merging, and sorting networks; zero-one principle; odd-even and bitonic sorting	11
4	Prefix accumulation; Ladner-Fischer circuit; BSP local work, communication, and synchronization	14
5	Parallel generalized linear recurrences and binary addition through prefix accumulation	17
6	LP, feasible and optimal points, standard and equational forms, fundamental theorem, simplex idea, and shortest paths as LP	20
7	Maximum flow as LP; augmenting paths; maximum-flow theorem; Ford-Fulkerson and its integral complexity	23
8	Dual LP in standard and equational forms; complementary slackness lemma; weak duality	26
9	Strong duality theorem and proof	28
10	Fractional minimum-cost cut as the maximum-flow dual; probabilistic integrality proof	30
11	Approximation algorithms; unweighted vertex cover; weighted LP rounding	33
12	Dual-feasibility and the efficient primal-dual weighted vertex-cover algorithm	35
13	Metric TSP; double-tree and Christofides-Serdyukov approximation algorithms	37
14	Randomized algorithms; matrix-product verification; Freivalds' algorithm	40
15	Polynomial identity testing and the Schwartz-Zippel lemma	42
16	Universal hash families and universality of multiply-add	44
17	Multiply-shift and its $2/m$ universality bound	46
18	Perfect hashing and two-level FKS perfect hashing	47
19	Collision resolution by chaining and linear probing; simplified analysis	49
20	Binary probing and its simplified expected-time analysis	51
21	SAT and k -SAT; deterministic SCC algorithm for 2-SAT	54

Item	Required topic	Page
22	Random-walk algorithm for 2-SAT	56
23	Schoning's randomized algorithm for 3-SAT	58
24	Primality testing and the Miller-Rabin algorithm	60
25	Online algorithms; scooter rental; caching and the marking algorithm	63
26	MST; Boruvka, Jarnik-Prim, and Kruskal algorithms	66

Reference Sheets and Further Reading

B.1 Principal bounds at a glance

Method	Bound	Qualification
Direct DFT	$\Theta(n^2)$ work	suitable primitive root
FFT / inverse FFT	$\Theta(n \log n)$ work	smooth transform length
Karatsuba	$\Theta(n^{\log_2 3})$	coefficient operations
FFT polynomial product	$\mathcal{O}(n \log n)$	zero padding, transform length $\Theta(n)$
Odd-even or bitonic merge	$\mathcal{O}(n \log n)$ size, $\mathcal{O}(\log n)$ depth	comparison network
Batcher sorting networks	$\mathcal{O}(n \log^2 n)$ size and $\mathcal{O}(\log^2 n)$ depth	oblivious sorting
Ladner-Fischer scan	$\mathcal{O}(n)$ size, $\mathcal{O}(\log n)$ depth	associative operator
BSP scan	comp, comm = $\mathcal{O}(n/p)$, sync = $\mathcal{O}(1)$	$n \geq p^2$
Parallel recurrence/addition	$\mathcal{O}(n)$ size, $\mathcal{O}(\log n)$ depth	associative affine composition
Ford-Fulkerson	$\mathcal{O}(E F^*)$	integral capacities
Maximal-matching cover	factor 2, $\mathcal{O}(V + E)$	unweighted
LP-rounded cover	factor 2, polynomial time	nonnegative weights
Primal-dual cover	factor 2, $\mathcal{O}(V + E)$	nonnegative weights
Double-tree TSP	factor 2	metric complete graph
Christofides-Serdyukov	factor 3/2	metric complete graph
Freivalds, k rounds	$\mathcal{O}(kn^2)$, error 2^{-k}	one-sided
Schwartz-Zippel	false zero at most $d/ S $	total degree d
Multiply-add	collision at most $1/m$	prime field reduction
Multiply-shift	collision at most $2/m$	odd word multiplier
FKS perfect hashing	$\mathcal{O}(s)$ space/build expected, $\mathcal{O}(1)$ query worst case	static set
Chaining	expected $\mathcal{O}(1 + \alpha)$	universal hashing
Linear probing	expected $\mathcal{O}(1)$	ideal hashing, $\alpha \leq 1/2$
Binary probing	expected $\mathcal{O}(1)$	ideal hashing, $\alpha \leq 1/4$
Deterministic 2-SAT	$\mathcal{O}(n + m)$	SCC computation
Random-walk 2-SAT	$\mathcal{O}(mn^2)$, success at least $1/2$	one run
Schoning 3-SAT	$\mathcal{O}^*((4/3)^n)$	randomized, one-sided
Miller-Rabin, k rounds	$\mathcal{O}(kn^3)$ bit operations, error 2^{-k}	schoolbook arithmetic
Marking paging	expected $\mathcal{O}(\log k)$ -competitive	oblivious adversary
Boruvka	$\mathcal{O}(E \log V)$	connected undirected graph
Prim, binary heap	$\mathcal{O}((V + E) \log V)$	adjacency lists

Method	Bound	Qualification
Kruskal	$\mathcal{O}(E \log E)$	sorting plus DSU

B.2 Recurring proof templates

Algebraic diagonalization. Prove a transform identity on evaluation points, perform easy componentwise work, then invert the transform.

Divide and conquer. State the exact decomposition, prove the combine formula, and solve the recurrence. FFT and Karatsuba differ because their numbers of recursive products differ.

Zero-one reduction. Show every comparator commutes with a monotone threshold map; prove correctness only on binary inputs.

Associative scan. Package a recurrence state as a composable transformation, prove closure and associativity, then invoke prefix scan.

Exchange argument. Insert a proposed safe edge into an optimum, locate the unique cycle, and exchange an edge without increasing cost.

Primal-dual certificate. Construct feasible solutions on both sides, use weak duality for a bound, and force equality through complementary slackness or a controlled approximate slackness condition.

One-sided random test. Correct instances always pass; prove that a fixed nonzero defect survives a random projection or evaluation with noticeable probability; amplify independently.

Random threshold. Express a fractional quantity as the probability of an integral event, equate expected integral cost with fractional cost, and select an outcome no worse than the average.

Competitive phases. Bound online cost per phase and charge a lower bound to the offline optimum over one or two neighboring phases.

B.3 Pseudocode and model conventions

- Arrays are zero-indexed; ranges written $0..n - 1$ include both endpoints unless a half-open interval is explicitly displayed.
- ∞ is a sentinel larger than any finite value, not an integer that may overflow under arithmetic.
- A claimed constant-time arithmetic operation belongs to the stated algebraic or word-RAM model. Bit complexity is stated separately for Miller-Rabin.
- Random choices are mutually independent unless explicitly coupled.
- Graph running times assume adjacency lists. Parallel size means work; parallel depth means unlimited-processor critical-path length.
- Approximation ratios apply only to feasible outputs. Randomized and online guarantees specify the source of probability and the adversarial model.

B.4 Oral-exam checklist

For each examination question, answer in this order:

1. state the input, output, assumptions, and computation model;
2. give the key definition or optimization formulation;
3. explain the representation change or central invariant;
4. state the algorithm in enough detail to reconstruct it;
5. prove feasibility and correctness, naming the certificate at termination;
6. derive work, depth, memory, approximation, probability, or competitive bounds rather than merely quoting them;
7. give one small example and one failure mode if an assumption is removed.

B.5 Selected references

This volume is self-contained for the supplied programme. The first source below is the course material on which its selection and emphasis are based; the remaining books provide broader treatments.

Bibliography

- [1] A. V. Tiskin, *Mathematical Foundations of Algorithms*, Spring 2026 lecture slides, Faculty of Mathematics and Computer Science, Saint Petersburg State University, supplied course material.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
- [3] J. JáJá, *An Introduction to Parallel Algorithms*, Addison-Wesley, 1992.
- [4] R. Motwani and P. Raghavan, *Randomized Algorithms*, Cambridge University Press, 1995.
- [5] V. V. Vazirani, *Approximation Algorithms*, Springer, 2001.
- [6] D. P. Williamson and D. B. Shmoys, *The Design of Approximation Algorithms*, Cambridge University Press, 2011.
- [7] A. Borodin and R. El-Yaniv, *Online Computation and Competitive Analysis*, Cambridge University Press, 1998.
- [8] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows*, Prentice Hall, 1993.

Closing Perspective

The course joins techniques that at first look unrelated. Fourier transforms turn multiplication into pointwise arithmetic; prefix circuits turn dependence into an associative scan; dual variables turn upper bounds into certificates; random choices turn algebraic identities into fast tests; competitive analysis compares decisions made without a future to an omniscient optimum. In every case, the decisive step is the same: identify a representation in which the desired global property becomes local, composable, and provable.